National Forensic Sciences University
Delhi Campus
(An Institute of National Importance)

PROJECT REPORT

ON

"A Hybrid Approach to DNS Security: Combining Machine Learning-Based Threat Detection with Configuration Analysis and Remediation Guidance"

Submitted To

School of Cybersecurity & Digital Forensics
National Forensic Sciences University

For partial fulfilment for the award of degree

B.Tech.-M.Tech. Computer Science & Engineering

In

Cybersecurity

Submitted By

Rupam Barui
(102CTBMCS2122002)

Under the Supervision of

Mr. Aditya Pratap Singh
Assistant Professor
School of Cybersecurity & Digital Forensics
National Forensic Sciences University

National Forensic Sciences University,
Delhi Campus, New Delhi – 110085, India

April 2026

DECLARATION

I hereby declare that the thesis entitled "**A Hybrid Approach to DNS Security: Combining Machine Learning-Based Threat Detection with Configuration Analysis and Remediation Guidance**" is a research work done by me and no part of the thesis has been presented earlier and will be presented for any degree, diploma or similar title at any other institute/university.

Date: April 25, 2026

Place: New Delhi, Delhi, India

Rupam Barui
102CTBMCS2122002
B.Tech.-M.Tech. Computer Science & Engineering
2026

ORIGINALITY REPORT CERTIFICATE

I certify that

a. The work contained in the dissertation is original and has been done by myself under the supervision of my supervisor.

b. The work has not been submitted to any other Institute for any degree or diploma.

c. I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.

d. Whenever I have used materials (data, theoretical analysis, and text) from other sources, I have given due credit to them by citing them in the text of the dissertation and giving their details in the references.

e. Whenever I have quoted written materials from other sources, due credit is given to the sources by citing them.

f. From the plagiarism test, it is found that the similarity index of the whole dissertation is within 10% and single paper is less than 10% as per the university guidelines.

Date: April 25, 2026

Place: New Delhi, Delhi, India

Rupam Barui

Enroll. No.: 102CTBMCS2122002

Forwarded by

(Mr. Aditya Pratap Singh)

Date: April 25, 2026

CERTIFICATE

This is to certify that the work contained in the dissertation entitled "**A Hybrid Approach to DNS Security: Combining Machine Learning-Based Threat Detection with Configuration Analysis and Remediation Guidance**", submitted by **Rupam Barui** holding the enrollment number: **102CTBMCS2122002** for the award of the degree of **B.Tech.-M.Tech. Computer Science & Engineering** in Cybersecurity to the **National Forensic Sciences University**, **Delhi Campus**, is a record of bonafide research works carried out by him under my supervision and guidance.

Date: April 25, 2026
Place: New Delhi, Delhi, India

Mr. Aditya Pratap Singh
Assistant Professor
School of Cybersecurity & Digital Forensics
National Forensic Sciences University
Delhi Campus, New Delhi – 110085, India

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who have contributed to the successful completion of this dissertation.

First and foremost, I am deeply grateful to my dissertation supervisor, **Mr. Aditya Pratap Singh**, Assistant Professor, School of Cybersecurity & Digital Forensics, National Forensic Sciences University, for his invaluable guidance, constant encouragement, and constructive feedback throughout the course of this research. His expertise and insights have been instrumental in shaping this work.

I extend my heartfelt thanks to the **Head of the Department** and all the **faculty members** of the School of Cybersecurity & Digital Forensics for their academic support and for providing a stimulating research environment. Their teachings have laid a strong foundation for this work.

I am also thankful to my **lab mates and peers** for their cooperation, technical discussions, and moral support during the challenging phases of this project. The collaborative spirit within the department has greatly enriched my learning experience.

I would like to acknowledge the **National Forensic Sciences University, Delhi Campus**, for providing the necessary infrastructure, resources, and facilities that made this research possible.

Finally, I owe my deepest gratitude to my **family** for their unwavering support, patience, and encouragement throughout my academic journey. Their belief in me has been a constant source of motivation.

With Sincere Regards,

Rupam Barui
102CTBMCS2122002
B.Tech.-M.Tech. Computer Science &
Engineering
2026

ABSTRACT

The Domain Name System (DNS) forms a critical backbone of the modern internet, yet it remains a frequent target for abuse, misconfiguration, and malicious exploitation. This dissertation presents **DNS Guard**, a hybrid domain threat intelligence platform that combines machine learning-based threat detection with comprehensive DNS configuration analysis and actionable remediation guidance.

The system performs multi-dimensional analysis of any given domain, extracting a 40-feature vector encompassing lexical characteristics, DNS record structure, DNSSEC validation, and composite security indicators. An ensemble model combining a Random Forest classifier with a Multi-Layer Perceptron (MLP) neural network produces a 0–100 threat score, accompanied by SHAP-style feature importance explanations for interpretability.

Beyond scoring, DNS Guard integrates a suite of security intelligence modules including WHOIS analysis with change detection, Certificate Transparency log monitoring, passive DNS history with fast-flux detection, typosquatting detection against top global brands, subdomain enumeration, reverse IP lookup, and real-time blocklist checks via Spamhaus DBL and PhishTank. IP geolocation is visualised on an interactive SVG world map, and comprehensive PDF reports are generated for each scan.

The platform further incorporates a context-aware AI chatbot powered by a local Ollama LLM (Mistral), with a rule-based fallback engine covering key DNS security topics. A Retrieval - Augmented Generation (RAG) pipeline backed by a Qdrant vector database enables the chatbot to provide accurate, domain-specific security guidance.

The system is deployed as a containerised microservices architecture using Docker Compose, comprising a Next.js 14 frontend, a FastAPI backend with Celery task queuing, a dedicated AI/ML service, PostgreSQL for persistence, and Redis as the message broker. Evaluation demonstrates that DNS Guard effectively identifies malicious, misconfigured, and suspicious domains with high accuracy, offering a practical and extensible tool for DNS security monitoring and threat intelligence.

Keywords: DNS Security, Threat Intelligence, Machine Learning, DNSSEC, Typosquatting, Fast-Flux Detection, Phishing Detection, Natural Language Processing, RAG, Cybersecurity.

Contents

Declaration i

Originality Report Certificate ii

Certificate iii

Acknowledgement iv

Abstract v

Abbreviations viii

1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope 3
1.5 Report Organisation 3
2.1 Current / Existing System 5
2.1.1 Study of Current System 5
2.1.2 Problem and Weakness of Current System 6
2.2 Requirements of New System 6
2.3 Feasibility Study 7
2.3.1 Technical Feasibility 7
2.3.2 Operational Feasibility 8
2.4 Tools and Technology Required 8
2.4.1 Frontend: Next.js 14 and Tailwind CSS 8
2.4.2 Backend: FastAPI and Celery 9
2.4.3 Database and Storage: PostgreSQL, Redis, and Qdrant 9
2.4.4 Machine Learning: scikit-learn 10
2.4.5 DNS Resolution: dnspython 10
2.4.6 PDF Generation: ReportLab 10
2.4.7 AI and LLM: Ollama and Mistral 11
2.4.8 Containerisation: Docker and Docker Compose 11

3.1 Function of System 12
3.1.1 Use Case Diagram 12
3.1.2 Activity Diagram 13
3.1.3 Sequence Diagram 14
3.2 Data Modelling 15
3.2.1 Entity-Relationship Diagram 15
3.2.2 Class Diagram 16
3.3 Functional and Behavioural Modelling 18
3.3.1 Data Flow Diagram 18
3.3.2 Data Dictionary 19
4.1 Implementation Environment 23
4.1.1 Model Used in Developing 23
4.1.2 Software Prototyping Types 24
4.2 Coding Standard 24
4.3 Laboratory Setup 25
4.4 Tools and Technology Used 25
4.5 Screenshots / Snapshots 26
5.1 Advantages and Unique Features 29
5.2 Results and Discussions 30
5.3 Future Scope of Work 31

Conclusion **32**

References **34**

A API Reference **I**

B Machine Learning Model Details **III**

C Sample Scan Output **VI**

D Environment Variables and Configuration **VIII**

E Installation and Deployment Guide **X**

F Sample Generated PDF Report **XII**

List of Publications / Online References **XVI**

List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CT	Certificate Transparency
DBL	Domain Block List
DKIM	DomainKeys Identified Mail
DMARC	Domain-based Message Authentication, Reporting and Conformance
DNS	Domain Name System
DNSKEY	DNS Key Record
DNSSEC	Domain Name System Security Extensions
DS	Delegation Signer
FYP	Final Year Project
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IP	Internet Protocol
LLM	Large Language Model
ML	Machine Learning
MLP	Multi-Layer Perceptron
MX	Mail Exchange
NLP	Natural Language Processing
NS	Name Server
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
RF	Random Forest
RRSIG	Resource Record Signature
SHAP	SHapley Additive exPlanations
SPF	Sender Policy Framework

Abbreviation	Full Form
SSL	Secure Sockets Layer
SVG	Scalable Vector Graphics
TLD	Top-Level Domain
TLS	Transport Layer Security
TXT	Text Record
URL	Uniform Resource Locator
WHOIS	(Protocol for querying domain registration information)

List of Tables

3.2 Data Dictionary 20
4.3 Laboratory Setup 25
4.4 Tools and Technologies Used 26

A.1 API Endpoint Reference I

B.1 ML Model Feature Vector (40 Features) III
B.2 ML Model Evaluation Metrics V

D.1 Environment Variables and Configuration VIII

List of Figures

3.1 Use Case Diagram for DNS Guard 13
3.2 Activity Diagram for Domain Analysis 14
3.3 Sequence Diagram for Domain Analysis Request 15
3.4 Entity-Relationship Diagram for DNS Guard Database 16
3.5 Class Diagram for DNS Guard Backend 17
3.6 Data Flow Diagram – Level 0 (Context Diagram) 18
3.7 Data Flow Diagram – Level 1 19
4.8 DNS Guard – Home Page with Search Interface 26
4.9 DNS Guard – Analysis Dashboard (Part 1: Threat Score and DNS Records) . . 27
4.10 DNS Guard – Analysis Dashboard (Part 2: WHOIS and Certificate Transparency) 27
4.11 DNS Guard – Analysis Dashboard (Part 3: Threat Intelligence and Geolocation) 28
4.12 DNS Guard – Light Mode Interface 28

Introduction

1.1 Background

The internet today connects billions of people, devices, and services across the world. At the heart of this connectivity is a system called the Domain Name System (DNS). DNS works like a phone book for the internet. When a user types a website address such as `www.example.com` into a browser, DNS translates that human-readable name into a numerical IP address that computers use to find and connect to the right server [1]. Without DNS, users would have to remember long strings of numbers to visit any website, which would make the internet very difficult to use.

Because DNS is so important, it has become a major target for attackers. Over the years, criminals have found many ways to misuse DNS to carry out attacks. Some attackers register domain names that look very similar to well-known brands in order to trick users into visiting fake websites. This is called typosquatting [2]. Others use DNS to spread malware, run phishing campaigns, or hide the location of their command-and-control servers. Fast-flux DNS is one such technique where attackers rapidly change the IP addresses linked to a domain to avoid being blocked [3].

At the same time, many organisations fail to properly configure their DNS records. Missing or incorrect SPF, DKIM, and DMARC records leave email systems open to spoofing attacks [4]. Domains without DNSSEC are vulnerable to cache poisoning, where an attacker can redirect users to a fake website without them knowing [5]. These misconfigurations are very common and often go unnoticed until an attack has already happened.

Security researchers and network administrators need tools that can quickly check a domain for all of these issues in one place. Currently, most available tools only check one or two things at a time. There is a clear need for a platform that brings together DNS analysis, threat intelligence, machine learning-based scoring, and clear guidance on how to fix problems.

1.2 Problem Statement

Despite the critical role that DNS plays in internet infrastructure, there is a lack of accessible and comprehensive tools that can analyse a domain from multiple security angles at once. Existing solutions tend to be either too narrow in scope, too expensive for individual researchers

and small organisations, or too technical for non-expert users to understand.

The specific problems this project addresses are as follows:

- DNS abuse, including phishing, typosquatting, and fast-flux attacks, continues to grow every year [6].
- Many domain owners do not know whether their DNS configuration is correct or secure.
- There is no single open tool that combines DNS record analysis, WHOIS data, certificate transparency, passive DNS history, threat intelligence feeds, and machine learning-based risk scoring in one platform.
- When a domain is flagged as suspicious, users often do not receive clear guidance on what the problem is or how to fix it.

This project aims to solve these problems by building DNS Guard, a web-based platform that performs a full security analysis of any domain and presents the results in a way that is easy to understand.

1.3 Objectives

The main objectives of this project are:

1. To build a platform that can analyse any domain for DNS misconfigurations, security weaknesses, and signs of malicious activity.
2. To develop a machine learning model that assigns a threat score between 0 and 100 to a domain based on its DNS records and lexical features.
3. To integrate multiple threat intelligence sources, including Spamhaus DBL and PhishTank, to check whether a domain appears on known blocklists.
4. To detect typosquatting by comparing a given domain against a list of top global brands using string similarity techniques.
5. To provide WHOIS analysis, certificate transparency monitoring, passive DNS history, subdomain enumeration, and reverse IP lookup as part of a single scan.
6. To include an AI-powered chatbot that can answer questions about DNS security and explain the results of a scan in plain language.
7. To generate a downloadable PDF report summarising all findings for a given domain.
8. To build the system as a set of containerised microservices so that it is easy to deploy and maintain.

1.4 Scope

This project covers the following areas:

- Analysis of domain names and their associated DNS records, including A, MX, NS, TXT, DNSKEY, RRSIG, and DS records.
- WHOIS data retrieval and comparison between scans to detect changes in domain registration.
- Certificate transparency log queries to monitor SSL/TLS certificates issued for a domain.
- Passive DNS history to identify historical IP addresses and detect fast-flux behaviour.
- Typosquatting detection using edit-distance algorithms against a curated list of top brands.
- Subdomain enumeration using both brute-force wordlists and certificate transparency logs.
- Reverse IP lookup to find other domains hosted on the same IP address.
- Threat intelligence checks against Spamhaus DBL and PhishTank.
- IP geolocation and visualisation on a world map.
- A machine learning ensemble model combining a Random Forest classifier and a Multi-Layer Perceptron (MLP) neural network for threat scoring.
- A context-aware chatbot using a local Large Language Model (LLM) with a Retrieval-Augmented Generation (RAG) pipeline.
- PDF report generation for each domain scan.

The project does not cover real-time network traffic analysis, deep packet inspection, or active exploitation testing. It is designed as a passive analysis and intelligence tool.

1.5 Report Organisation

The rest of this report is organised as follows:

- **Chapter 2 – Literature Survey** reviews existing work on DNS security, threat detection, machine learning for cybersecurity, and related tools.
- **Chapter 3 – Design** covers the system architecture, the analysis of requirements, the design methodology, and the implementation strategy.

- **Chapter 4 – Implementation** describes how each component of the system was built, including the frontend, backend, AI service, and supporting infrastructure.
- **Chapter 5 – Summary of Results and Future Scope** presents the results of testing and evaluation, and discusses possible improvements and extensions.
- **Chapter 6 – Conclusion** summarises the work done and the key findings of the project.

Literature Survey

2.1 Current / Existing System

2.1.1 Study of Current System

Several tools and platforms exist today that help security researchers and network administrators check the security of a domain. Each of these tools focuses on a specific area of DNS security or threat intelligence.

VirusTotal [7] is one of the most widely used online tools for checking whether a domain, URL, or file has been flagged as malicious. It aggregates results from over 70 antivirus engines and blocklists. However, it does not provide detailed DNS record analysis, DNSSEC validation, or configuration guidance.

MXToolbox [8] is a popular tool for checking DNS records, MX configuration, and email security settings such as SPF, DKIM, and DMARC. It is useful for diagnosing email delivery problems but does not provide a threat score or machine learning-based analysis.

Shodan [9] is a search engine for internet-connected devices. It can be used to find information about the servers behind a domain, including open ports and running services. It is a powerful tool for security professionals but requires technical knowledge to use effectively.

SecurityTrails [10] provides historical DNS data, WHOIS records, and subdomain information. It is a good source of passive DNS intelligence but is a paid service with limited free access.

DNSdumpster [11] is a free tool for subdomain enumeration and DNS reconnaissance. It is useful for mapping the attack surface of a domain but does not provide threat scoring or remediation advice.

PhishTank [12] and **Spamhaus** [13] maintain community-driven and professionally managed blocklists of phishing and spam domains. These are widely used as threat feeds but are not standalone analysis platforms.

Research in the area of DNS threat detection has also produced several academic tools. Antonakakis et al. proposed EXPOSURE [14], a system that uses passive DNS data and machine learning technology to detect malicious domains. Bilge et al. introduced EXPOSURE's successor, which used a larger feature set to improve detection accuracy of the same [15]. More recently, deep learning approaches have been applied to detect domain generation algorithms

(DGAs) used by malware [16].

2.1.2 Problem and Weakness of Current System

While the tools described above are useful individually, they all have significant limitations when considered as a complete solution for DNS security analysis.

- **Fragmentation:** No single free tool combines DNS record analysis, WHOIS data, certificate transparency, passive DNS, typosquatting detection, subdomain enumeration, reverse IP lookup, and threat intelligence in one place. Users must visit multiple tools and manually combine the results.
- **No unified threat score:** Most tools present raw data without summarising the overall risk level of a domain. This makes it difficult for non-expert users to quickly understand whether a domain is safe or dangerous.
- **No remediation guidance:** Tools like MXToolbox show what is wrong with a DNS configuration but do not explain why it matters or how to fix it in simple terms.
- **Cost:** Many of the most capable tools, such as SecurityTrails and Shodan, require paid subscriptions. This limits access for students, independent researchers, and small organisations.
- **No AI assistance:** None of the existing free tools include a conversational AI assistant that can answer questions about the results of a scan or explain DNS security concepts in plain language.
- **Limited explainability:** Academic machine learning models for DNS threat detection often do not explain which features contributed to a classification decision, making it hard to trust or act on the results [17].

2.2 Requirements of New System

Based on the study of existing systems and their weaknesses, the following requirements were identified for DNS Guard.

Functional Requirements:

- The system must accept any domain name as input and perform a full security analysis.
- It must check DNS records including A, MX, NS, TXT, DNSKEY, RRSIG, and DS.
- It must validate DNSSEC configuration and report whether it is properly set up.
- It must retrieve and display WHOIS data and detect changes between scans.

- It must query certificate transparency logs to find SSL/TLS certificates issued for the domain.
- It must retrieve passive DNS history and detect fast-flux behaviour.
- It must check the domain against Spamhaus DBL and PhishTank blocklists.
- It must detect typosquatting by comparing the domain against a list of top brands.
- It must enumerate subdomains using brute-force and certificate transparency methods.
- It must perform reverse IP lookup to find co-hosted domains.
- It must produce a threat score between 0 and 100 using a machine learning model.
- It must provide feature importance information alongside the threat score.
- It must include an AI chatbot that can answer DNS security questions.
- It must generate a downloadable PDF report for each scan.

Non-Functional Requirements:

- The system must be fast enough to return results within a reasonable time for a typical domain.
- The user interface must be clean and easy to use for both technical and non-technical users.
- The system must support both dark and light display modes.
- The system must be deployable using Docker Compose with minimal setup.
- The system must store scan history in a database for future reference.

2.3 Feasibility Study

2.3.1 Technical Feasibility

The technologies required to build DNS Guard are all mature, well-documented, and freely available. Python has a rich ecosystem of libraries for DNS resolution, WHOIS lookup, machine learning, and web development. The `dnspython` library [18] provides reliable DNS query capabilities. The `scikit-learn` library [19] provides the tools needed to build and train the machine learning model. FastAPI [20] is a modern Python web framework that is well suited for building high-performance APIs. Next.js [21] provides a solid foundation for building the frontend with server-side rendering and a good developer experience.

The use of Docker and Docker Compose [22] makes it straightforward to package all components of the system into containers that can be run on any machine. PostgreSQL [23] and Redis [24] are both widely used and reliable choices for data storage and task queuing respectively. Qdrant [25] is a purpose-built vector database that is well suited for the RAG pipeline used by the chatbot.

Ollama [26] provides a simple way to run open-source LLMs locally, which avoids the cost and privacy concerns of using cloud-based AI services. The Mistral model [27] is a capable open-source LLM that runs well on consumer hardware.

Overall, the technical feasibility of the project is high. All required components are available, well-supported, and have been used successfully in similar projects.

2.3.2 Operational Feasibility

DNS Guard is designed to be easy to deploy and use. The entire system can be started with a single `docker compose up` command, which means that users do not need to manually install or configure any dependencies. The web interface is accessible through a standard browser and does not require any special software on the client side.

The system is intended for use by security researchers, network administrators, students, and anyone who needs to check the security of a domain. The interface is designed to present results clearly, with visual indicators such as colour-coded threat scores and a world map for IP geolocation. The AI chatbot provides an additional layer of support for users who are not familiar with DNS security concepts.

From an operational standpoint, the system is feasible for use in an academic or small-organisation setting. For larger-scale deployment, additional work would be needed to handle high volumes of concurrent requests and to keep threat intelligence feeds up to date automatically.

2.4 Tools and Technology Required

This section describes each tool and technology used in DNS Guard, explains what it does, why it was chosen over alternatives, and how it fits into the overall system.

2.4.1 Frontend: Next.js 14 and Tailwind CSS

Next.js 14 [21] is a React-based web framework developed by Vercel. It was chosen as the frontend framework for DNS Guard for several reasons. First, it supports server-side rendering (SSR) and the App Router, which means pages can be rendered on the server before being sent to the browser. This improves page load speed and makes the application more responsive. Second, Next.js has built-in support for API routes, which were used to create proxy endpoints

that forward requests from the browser to the backend without exposing the backend URL directly to the user. Third, Next.js has a large community, good documentation, and is widely used in production applications, which made it a reliable choice for this project.

An alternative would have been to use plain React with a separate Express.js server, but this would have required more configuration and would not have provided the same level of built-in optimisation.

Tailwind CSS [28] is a utility-first CSS framework. Instead of writing custom CSS classes, developers apply small, single-purpose utility classes directly in the HTML. This approach was chosen because it speeds up development significantly and makes it easy to build a consistent and responsive design. Tailwind also has built-in support for dark mode, which was used to implement the dark and light theme toggle in DNS Guard. The alternative, writing custom CSS from scratch, would have taken much longer and been harder to maintain.

2.4.2 Backend: FastAPI and Celery

FastAPI [20] is a modern Python web framework for building APIs. It was chosen for both the main backend and the AI service because it is very fast, supports asynchronous request handling, and automatically generates interactive API documentation. FastAPI uses Python type hints to validate request and response data, which reduces bugs and makes the code easier to read. Compared to Flask, which is another popular Python web framework, FastAPI is significantly faster and has better support for async operations, which is important when making multiple DNS queries and external API calls at the same time.

Celery [29] is a distributed task queue for Python. DNS analysis involves many steps, including DNS resolution, WHOIS lookup, certificate transparency queries, and machine learning scoring. Running all of these steps synchronously in a single HTTP request would make the API very slow and could cause timeouts. Celery solves this by allowing these tasks to be run in the background by worker processes. The frontend polls for results while the workers process the analysis. Redis is used as the message broker that passes tasks from the API to the workers. This architecture makes the system much more scalable and responsive.

2.4.3 Database and Storage: PostgreSQL, Redis, and Qdrant

PostgreSQL [23] is a powerful open-source relational database. It was chosen to store scan results, WHOIS history, and other persistent data because it is reliable, supports complex queries, and handles structured data well. Compared to SQLite, which is simpler but not suitable for concurrent access, PostgreSQL is a better choice for a multi-service application where the backend and workers may read and write data at the same time.

Redis [24] is an in-memory data store that is used in DNS Guard as the message broker for Celery. When a user submits a domain for analysis, the backend creates a Celery task and sends it to Redis. The Celery worker picks up the task from Redis and processes it. Redis was

chosen because it is fast, lightweight, and is the most commonly used broker with Celery. It is also used to cache frequently accessed data to reduce the number of repeated database queries.

Qdrant [25] is a vector database designed for storing and searching high-dimensional vector embeddings. It is used in DNS Guard to power the RAG pipeline for the AI chatbot. DNS security documents and knowledge base articles are converted into vector embeddings and stored in Qdrant. When a user asks the chatbot a question, the question is also converted into a vector and Qdrant finds the most relevant documents. These documents are then passed to the LLM as context so it can give an accurate and relevant answer. Qdrant was chosen over alternatives such as FAISS because it runs as a standalone service, supports filtering, and is easier to integrate into a microservices architecture.

2.4.4 Machine Learning: scikit-learn

scikit-learn [19] is a widely used Python library for machine learning. It provides implementations of many standard algorithms including Random Forest, which is one of the two models used in the DNS Guard threat scoring ensemble. scikit-learn was chosen because it is well-documented, easy to use, and produces reliable results. It also provides tools for feature extraction, model evaluation, and cross-validation, all of which were used during the development of the threat scoring model. The MLP neural network was also implemented using scikit-learn's `MLPClassifier`, which provided a good balance between performance and simplicity compared to using a full deep learning framework such as TensorFlow or PyTorch.

2.4.5 DNS Resolution: dnspython

dnspython [18] is a Python library for making DNS queries. It was used in the backend to resolve DNS records for a given domain, including A, MX, NS, TXT, DNSKEY, RRSIG, and DS records. dnspython was chosen because it supports a wide range of DNS record types, handles DNSSEC validation, and is actively maintained. It is the standard choice for DNS work in Python and has no significant alternatives that offer the same level of functionality.

2.4.6 PDF Generation: ReportLab

ReportLab [30] is a Python library for generating PDF documents programmatically. It was used to create the downloadable PDF reports in DNS Guard. ReportLab was chosen because it gives full control over the layout and content of the PDF, including tables, charts, and formatted text. An alternative would have been to use a headless browser to render an HTML page as a PDF, but this approach is heavier and slower. ReportLab produces PDFs directly from Python code, which is faster and more suitable for a server-side API.

2.4.7 AI and LLM: Ollama and Mistral

Ollama [26] is a tool that makes it easy to run open-source large language models locally on a personal computer or server. It was used in DNS Guard to run the **Mistral 7B** model [27], which powers the AI chatbot. Running the LLM locally has several advantages over using a cloud-based API such as OpenAI. It is free to use, does not send user data to a third party, and does not require an internet connection once the model is downloaded. Mistral 7B was chosen because it is one of the best-performing open-source models of its size, offering a good balance between response quality and the hardware resources required to run it [27]. A rule-based fallback engine was also implemented for cases where Ollama is not available.

2.4.8 Containerisation: Docker and Docker Compose

Docker [22] is a platform for building and running applications inside containers. A container packages an application together with all its dependencies so that it runs the same way on any machine. Docker was used to containerise all six components of DNS Guard: the frontend, the backend, the AI service, PostgreSQL, Redis, and Qdrant. **Docker Compose** was used to define and manage all these containers together in a single configuration file. This means the entire system can be started with one command, which makes it very easy to set up and deploy. Without Docker, a user would need to manually install Python, Node.js, PostgreSQL, Redis, and Qdrant separately, which would be time-consuming and error-prone.

Design: Analysis, Design Methodology and Implementation Strategy

3.1 Function of System

DNS Guard is a web-based domain threat intelligence platform. Its primary function is to accept a domain name from the user and perform a comprehensive security analysis across multiple dimensions. The system checks DNS records, validates DNSSEC configuration, retrieves WHOIS data, queries certificate transparency logs, checks passive DNS history, detects typosquatting, enumerates subdomains, performs reverse IP lookup, checks threat intelligence blocklists, and produces a machine learning-based threat score. All results are presented in a single dashboard and can be exported as a PDF report. An AI chatbot is also available to answer questions about the results and about DNS security in general.

The system is built as a set of microservices. The frontend communicates with the backend through a set of REST API endpoints. The backend delegates long-running analysis tasks to Celery workers. The AI service handles threat scoring and chatbot requests separately. All services share a common PostgreSQL database and communicate through Redis.

3.1.1 Use Case Diagram

The use case diagram shows all the ways a user can interact with DNS Guard and what happens inside the system as a result of each interaction. There are two types of actors in this diagram. The first is the **User**, who is the person using the web interface. The second group consists of **External Services**, which are the third-party systems that DNS Guard queries during an analysis, such as DNS servers, WHOIS servers, certificate transparency logs, and blocklist providers.

The user can perform five main actions. The most important one is submitting a domain for analysis. When this happens, the system internally triggers eight sub-processes: DNS record checking, WHOIS lookup, certificate transparency querying, threat intelligence checking, machine learning threat scoring, typosquatting detection, subdomain enumeration, and reverse IP lookup. Each of these sub-processes runs as part of the same analysis job. The user can also view the results on the analysis dashboard, download a PDF report, use the AI chatbot to ask questions, and view their recent search history. The recent search history is stored in the

browser's local storage and does not require a server request.

This diagram is important because it defines the boundary of the system and makes clear what the system is responsible for and what it relies on external services to provide.

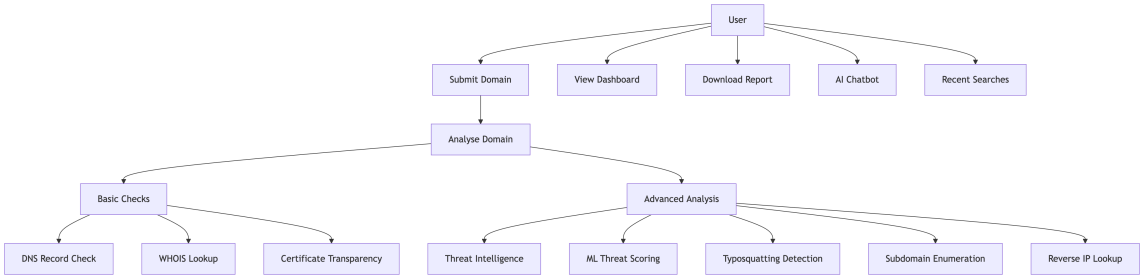

Figure 3.1: Use Case Diagram for DNS Guard

3.1.2 Activity Diagram

The activity diagram shows the step-by-step flow of actions that take place from the moment a user submits a domain to the moment the results are displayed on screen. It captures both the user-facing actions and the internal processing that happens behind the scenes.

The process begins when the user types a domain name into the search box and clicks the analyse button. The frontend validates the input and sends a POST request to the backend API with the domain name. The backend receives the request, creates a new Celery task, and immediately returns a task ID to the frontend. This is important because it means the API does not block while waiting for the analysis to finish. Instead, the frontend uses the task ID to poll the backend every few seconds and check whether the analysis is complete.

In the background, the Celery worker picks up the task from the Redis queue and begins running all the analysis modules. These modules run in parallel where possible to reduce the total time taken. The modules are: DNS resolution, WHOIS lookup, certificate transparency querying, passive DNS history retrieval, typosquatting detection, subdomain enumeration, reverse IP lookup, and threat intelligence checking. At the same time, the feature extractor builds a 40-feature vector from the DNS data and sends it to the AI service, which returns a threat score and feature importance values.

Once all modules have finished, the results are combined into a single result object and saved to the PostgreSQL database. The task is then marked as complete in Redis. On the next poll from the frontend, the backend returns the full results. The frontend renders the analysis dashboard, showing the threat score, DNS records, WHOIS data, certificate information, geolocation map, and all other findings.

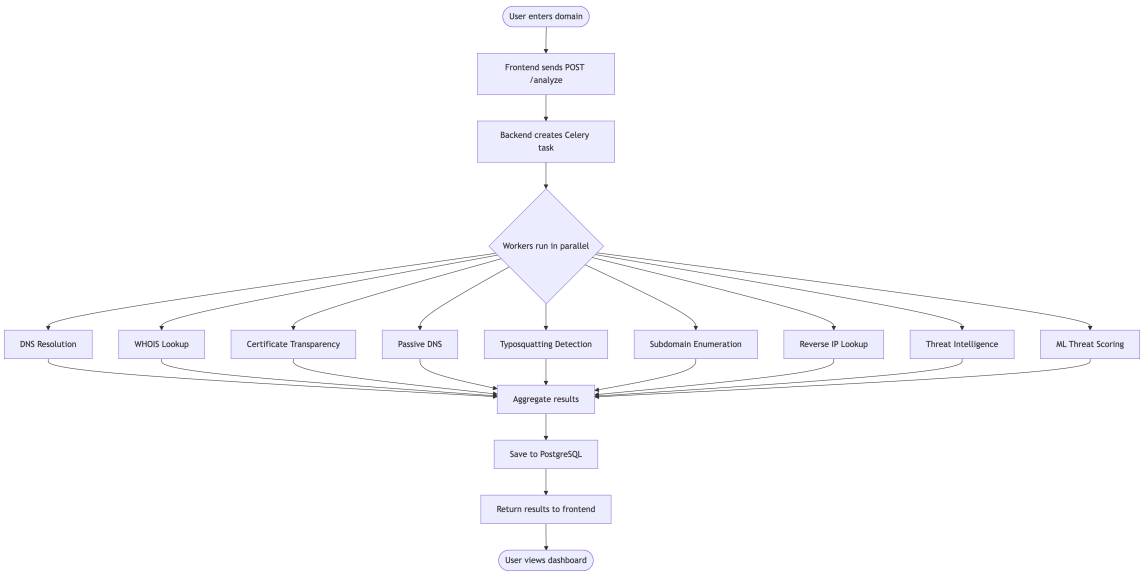

Figure 3.2: Activity Diagram for Domain Analysis

3.1.3 Sequence Diagram

The sequence diagram shows the exact order of messages passed between the different components of DNS Guard during a domain analysis. Unlike the activity diagram, which focuses on the flow of actions, the sequence diagram focuses on which component sends a message to which other component and in what order.

There are seven participants in this diagram: the User's browser, the Next.js frontend, the FastAPI backend, the Celery worker, the AI service, the PostgreSQL database, and Redis.

The sequence begins when the user submits a domain. The frontend sends a POST request to the backend's `/analyze` endpoint. The backend does not process the analysis itself. Instead, it pushes a task onto the Redis queue and immediately returns a task ID to the frontend. This keeps the API response fast and prevents the HTTP connection from timing out during a long analysis.

The frontend then enters a polling loop, sending a GET request to a endpoint on the API `/analyze/status/{task_id}` every few seconds. Meanwhile, the Celery worker picks up the task from Redis and begins the analysis. When it needs a threat score, it sends a POST request to the AI service's `/score` endpoint, passing the extracted feature vector. The AI service runs the ensemble model and returns the score and feature importance values. The worker saves all results to PostgreSQL and marks the task as complete in Redis.

On the next poll from the frontend, the backend finds that the task is complete, retrieves the results from PostgreSQL, and returns them to the frontend. The frontend then renders the full analysis dashboard for the user.

This sequence makes the system non-blocking and scalable. Multiple users can submit domains at the same time because each analysis runs as a separate Celery task in the background.

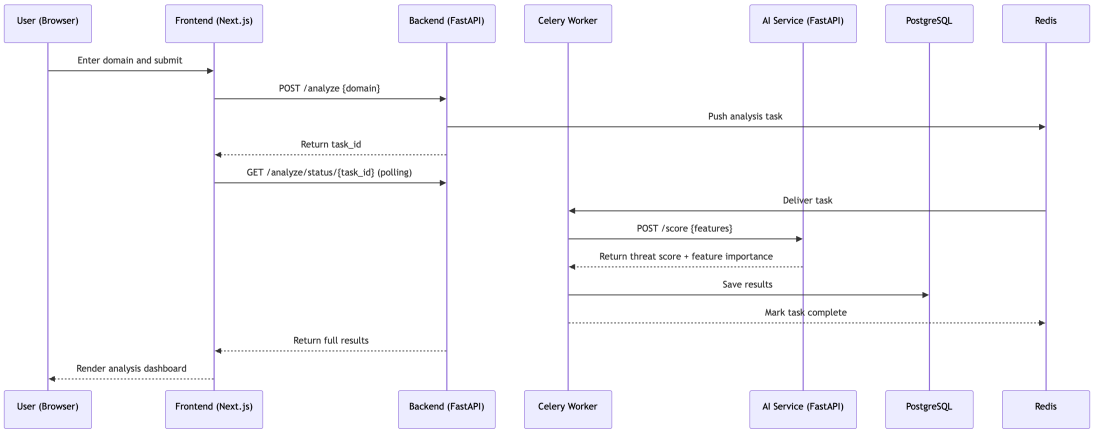

Figure 3.3: Sequence Diagram for Domain Analysis Request

3.2 Data Modelling

3.2.1 Entity-Relationship Diagram

The entity-relationship (ER) diagram shows the structure of the PostgreSQL database used by DNS Guard. It defines the main entities, their attributes, and the relationships between them.

The central entity is **Scan**, which represents a single analysis run for a domain. Every time a user submits a domain, a new Scan record is created. The Scan entity stores the time the scan was created and its current status (pending, running, or complete). Each Scan belongs to a **Domain** entity, which stores the domain name and the dates it was first and last seen in the system. A Domain can have many Scans over time, which allows the system to track how a domain's security posture changes between scans.

Each Scan is linked to multiple **DNS_Record** entries, one for each DNS record retrieved during the analysis. This includes A records, MX records, NS records, TXT records, and DNSSEC-related records. Storing individual records rather than a single JSON blob makes it easier to query and compare records across scans.

Each Scan also has exactly one **WHOIS_Record**, which stores the registration data returned by the WHOIS lookup, including the registrar name, creation date, expiry date, and the raw WHOIS response. A separate **WHOIS_Diff** entity is linked to the Domain (not the Scan) and records the differences detected between two consecutive WHOIS scans. This allows the system to alert users when registration details change, which can be a sign of domain hijacking.

Finally, each Scan has exactly one **Threat_Score** record, which stores the numerical score produced by the ML model and the feature importance values as a JSON object.

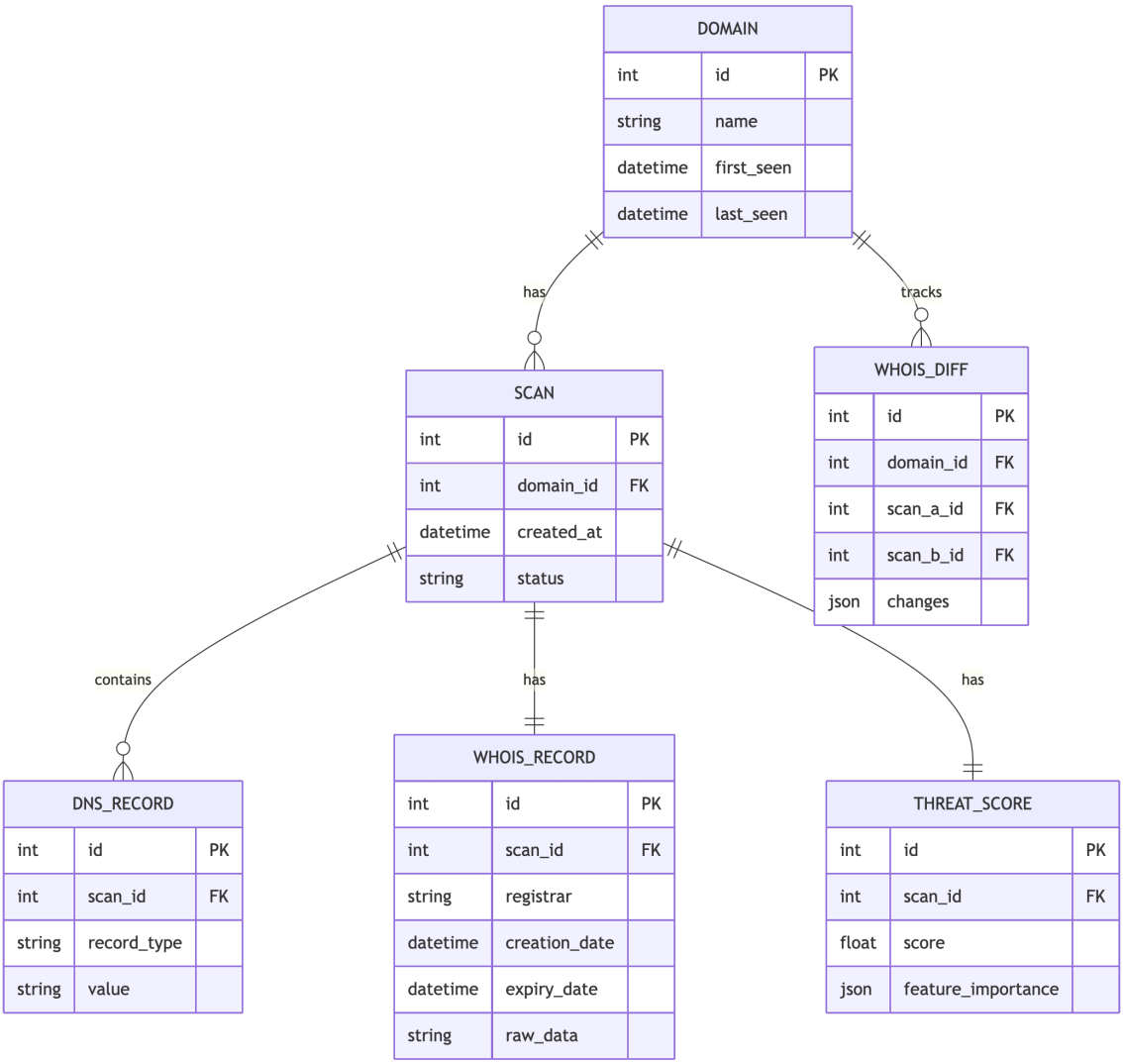

Figure 3.4: Entity-Relationship Diagram for DNS Guard Database

3.2.2 Class Diagram

The class diagram shows the main classes in the DNS Guard backend and how they are connected. It gives a clear picture of how the code is organised and how responsibility is divided between different parts of the system.

The central class is **AnalysisService**, which acts as the coordinator for the entire analysis process. When a Celery worker picks up an analysis task, it calls the `run()` method of `AnalysisService`, passing the domain name. This method then calls each of the individual service classes in turn and collects their results.

Each service class has a single responsibility. **DNSResolver** queries DNS servers and returns all DNS records for the domain. **WHOISLookup** retrieves registration data from WHOIS servers. **CertTransparency** queries the crt.sh certificate transparency log API. **PassiveDNS** retrieves historical IP resolution data. **TyposquatDetector** compares the domain against a list of top brands using edit-distance algorithms. **SubdomainEnum** discovers subdomains using both

brute-force wordlists and certificate transparency data. **ReverseIP** finds other domains hosted on the same IP address. **ThreatIntel** checks the domain against Spamhaus DBL and PhishTank. **ThreatScorer** sends the extracted feature vector to the AI service and returns the threat score and feature importance.

Each service class returns a typed result object. All of these result objects are combined by `AnalysisService` into a single **AnalysisResult** object, which is then serialised and saved to the database. This design makes the code easy to test and extend. Adding a new analysis module only requires creating a new service class and calling it from `AnalysisService`.

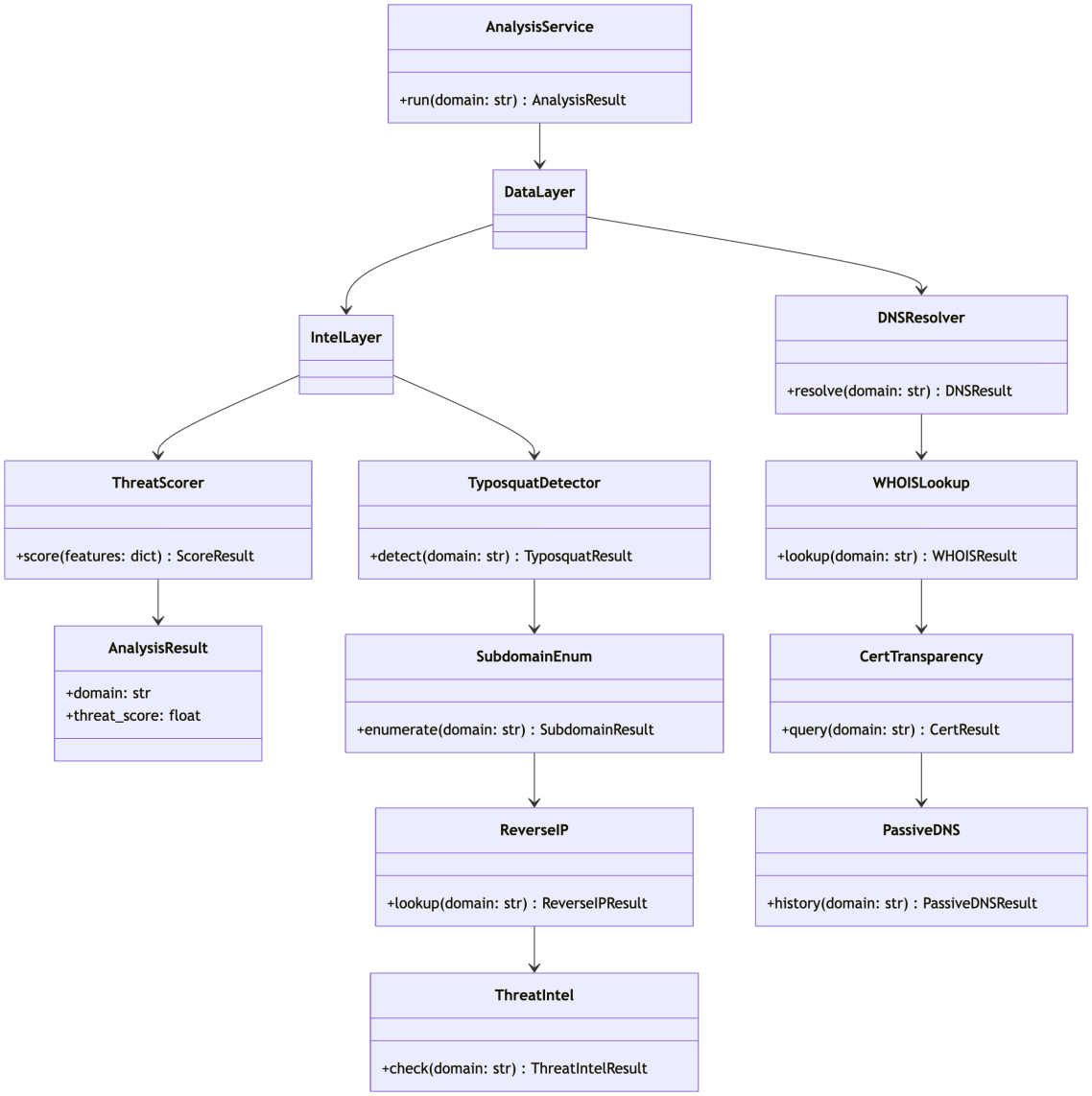

Figure 3.5: Class Diagram for DNS Guard Backend

3.3 Functional and Behavioural Modelling

3.3.1 Data Flow Diagram

The data flow diagram (DFD) shows how data enters, moves through, and leaves the DNS Guard system. Two levels of DFD are provided. Level 0 gives a high-level view of the system as a single process, and Level 1 breaks that process down into its main sub-processes.

Level 0 (Context Diagram): At this level, DNS Guard is shown as a single black box. The user provides a domain name as input and receives an analysis report as output. The system also communicates with four external data stores: DNS servers (to resolve DNS records), WHOIS servers (to retrieve registration data), the crt.sh certificate transparency log (to find SSL/TLS certificates), and the Spamhaus and PhishTank blocklist services (to check whether the domain is known to be malicious). Data flows in both directions between DNS Guard and each of these external services.

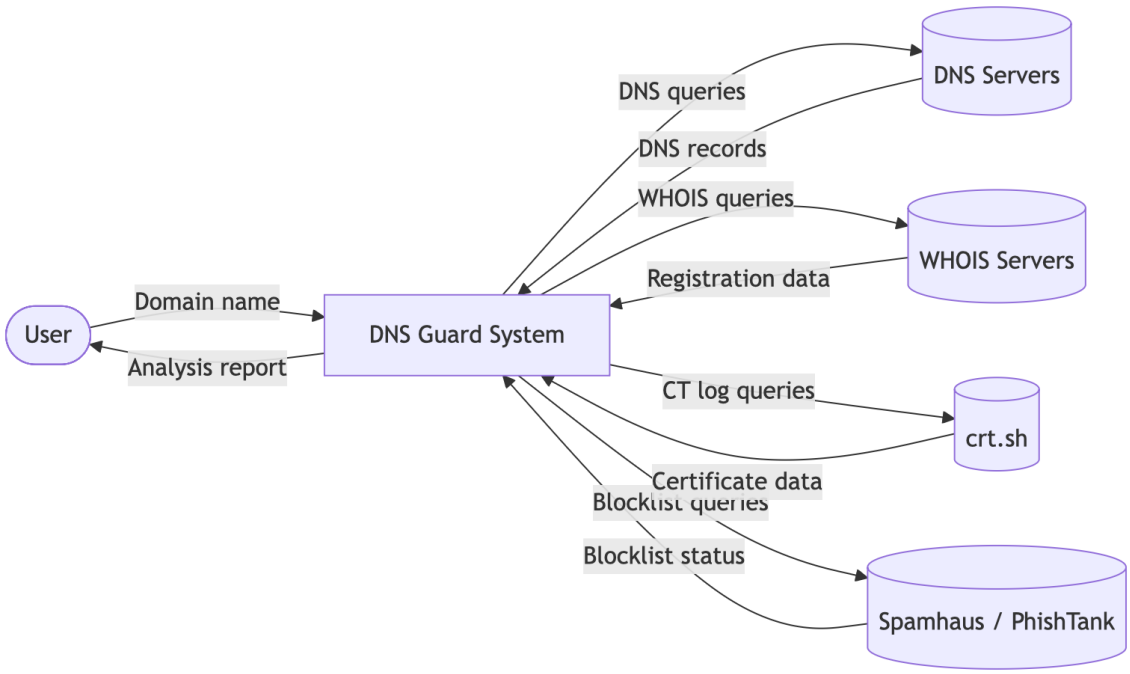

Figure 3.6: Data Flow Diagram – Level 0 (Context Diagram)

Level 1 (Detailed DFD): At this level, the internal processing of DNS Guard is broken down into six numbered processes. Process 1 (Data Collection) receives the domain name from the user and queries all external services to gather raw data. Process 2 (Feature Extraction) takes the raw DNS and WHOIS data and converts it into the 40-feature vector used by the machine learning model. Process 3 (ML Threat Scoring) receives the feature vector and returns a threat score and feature importance values. Process 4 (Threat Intelligence Check) runs in parallel with feature extraction and checks the domain against blocklists. Process 5 (Result Aggregation) combines the outputs of all previous processes into a single result object and

saves it to the PostgreSQL database. Process 6 (Report Generation) takes the aggregated result and produces a PDF report on request. The user receives both the dashboard data and the PDF from the system.

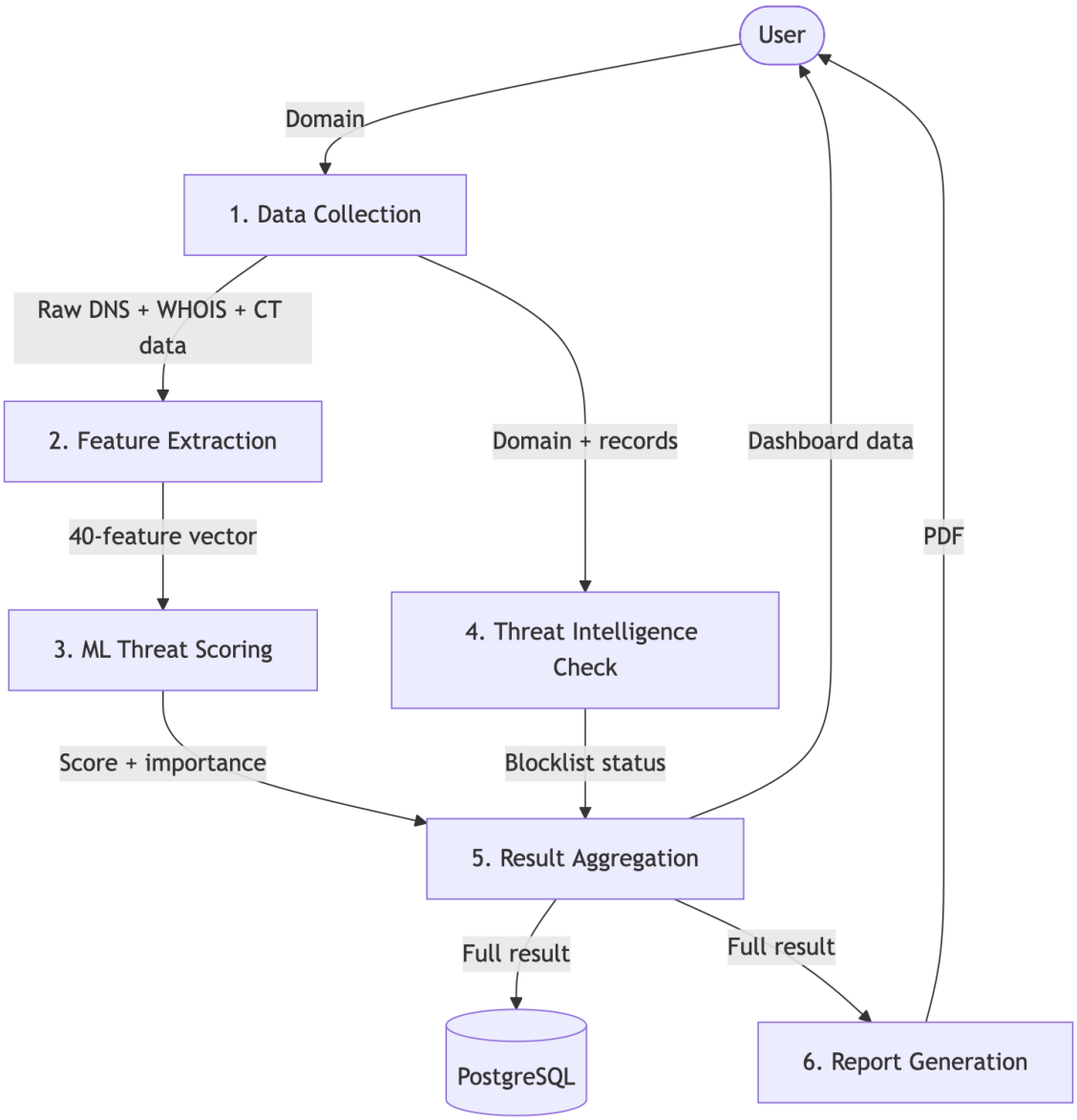

Figure 3.7: Data Flow Diagram – Level 1

3.3.2 Data Dictionary

The data dictionary below defines the key data elements used in DNS Guard. It lists the field name, data type, and a description of what each field stores and how it is used in the system.

Table 3.2: Data Dictionary

Field Name	Data Type	Description
`domain`	String	The domain name submitted by the user for analysis (e.g. `example.com`).
`scan_id`	Integer	A unique identifier assigned to each analysis scan. Used to retrieve results and track task status.
`created_at`	Datetime	The date and time when the scan was created. Stored in UTC.
`status`	String	The current status of the scan task. Possible values are `pending`, `running`, and `complete`.
`threat_score`	Float	A numerical risk score between 0 and 100 produced by the ML ensemble model. A score above 70 is considered high risk.
`feature_importance`	JSON	A dictionary mapping each of the 40 input features to its contribution to the threat score. Used to generate the SHAP-style bar chart.
`dns_records`	JSON	A list of DNS records retrieved for the domain. Each entry includes the record type (A, MX, NS, TXT, etc.), name, and value.
`dnssec_valid`	Boolean	Indicates whether the domain has a valid and complete DNSSEC configuration, including DNSKEY, RRSIG, and DS records.
`registrar`	String	The name of the domain registrar as returned by the WHOIS lookup (e.g. GoDaddy, Namecheap).
`creation_date`	Datetime	The date the domain was first registered, taken from WHOIS data. Used to flag newly registered domains.
`expiry_date`	Datetime	The date the domain registration is due to expire, taken from WHOIS data. Used to flag soon-to-expire domains.
`whois_diff`	JSON	A structured record of changes detected between two consecutive WHOIS scans of the same domain. Includes the field name, old value, and new value for each change.

Field Name	Data Type	Description
`cert_count`	Integer	The total number of SSL/TLS certificates found in certificate transparency logs for the domain. A sudden spike may indicate suspicious activity.
`has_wildcard_cert`	Boolean	Indicates whether any of the certificates found are wildcard certificates (e.g. `*.example.com`).
`passive_dns_ips`	JSON	A list of historical IP addresses that the domain has resolved to, retrieved from passive DNS data sources.
`fast_flux_detected`	Boolean	Indicates whether fast-flux behaviour was detected. Fast-flux is identified by a large number of different IP addresses in the passive DNS history over a short period.
`typosquat_matches`	JSON	A list of known brand domains that are similar to the submitted domain, along with their edit-distance similarity scores.
`subdomains`	JSON	A list of subdomains discovered through brute-force wordlist enumeration and certificate transparency log queries.
`reverse_ip_domains`	JSON	A list of other domain names found to be hosted on the same IP address as the submitted domain. A very large number of co-hosted domains may indicate shared malicious hosting.
`on_spamhaus_dbl`	Boolean	Indicates whether the domain appears on the Spamhaus Domain Block List, which covers spam, phishing, and malware domains.
`on_phishtank`	Boolean	Indicates whether the domain appears in the PhishTank database of verified phishing URLs.
`geo_locations`	JSON	A list of geolocation records for each IP address associated with the domain. Each entry includes country, city, latitude, and longitude.

Field Name	Data Type	Description
`spf_record`	String	The SPF TXT record found for the domain, if present. An absent or incorrectly configured SPF record leaves the domain open to email spoofing.
`dmarc_record`	String	The DMARC TXT record found for the domain, if present. DMARC builds on SPF and DKIM to provide a policy for handling unauthenticated email.
`dkim_record`	String	The DKIM TXT record found for the domain, if present. DKIM allows receiving mail servers to verify that an email was sent by an authorised server.

Implementation

4.1 Implementation Environment

4.1.1 Model Used in Developing

DNS Guard was developed using the **Incremental Development Model**. In this model, the system is built and delivered in small, working increments rather than all at once. Each increment adds new functionality on top of what was already built and tested. This approach was chosen because the project has many independent modules, such as DNS resolution, WHOIS lookup, certificate transparency, and machine learning scoring, each of which could be built, tested, and integrated one at a time.

The development went through the following increments:

1. **Increment 1 – Core Infrastructure:** Set up the Docker Compose environment with FastAPI backend, Next.js frontend, PostgreSQL, and Redis. Implemented the basic domain submission and task queuing flow using Celery.

2. **Increment 2 – DNS and WHOIS Analysis:** Implemented DNS record resolution using dnspython, DNSSEC validation, and WHOIS lookup. Added the results display on the frontend.

3. **Increment 3 – Threat Intelligence Modules:** Added certificate transparency querying, passive DNS history, typosquatting detection, subdomain enumeration, reverse IP lookup, and blocklist checks.

4. **Increment 4 – Machine Learning:** Built the feature extractor, trained the Random Forest and MLP ensemble model, and integrated the AI service with the backend.

5. **Increment 5 – AI Chatbot:** Set up the Qdrant vector database, built the RAG pipeline, and integrated the Ollama LLM with a rule-based fallback.

6. **Increment 6 – Reporting and Polish:** Added PDF report generation, the geolocation map, WHOIS diff detection, dark/light mode, and recent search history.

This incremental approach meant that a working version of the system was available at the end of each increment, which made it easy to test and get feedback throughout the development process.

4.1.2 Software Prototyping Types

During the development of DNS Guard, **evolutionary prototyping** was used alongside the incremental model. In evolutionary prototyping, the prototype is not thrown away after gathering feedback. Instead, it is continuously refined and improved until it becomes the final product. This was appropriate for this project because the requirements for some features, such as the threat scoring model and the chatbot, were not fully defined at the start and evolved as the project progressed.

For example, the initial version of the threat scoring model used only lexical features. After testing, it became clear that DNS record features and DNSSEC status were also important, so the feature set was expanded from 12 to 40 features. Similarly, the chatbot started as a simple keyword-matching rule engine and was later upgraded to use a full RAG pipeline with a local LLM.

4.2 Coding Standard

Consistent coding standards were followed throughout the project to keep the codebase readable and maintainable.

Python (Backend and AI Service):

- Code follows the **PEP 8** style guide [31], which defines conventions for indentation, naming, and line length.
- All functions and classes have docstrings explaining their purpose, parameters, and return values.
- Variable and function names use `snake_case`. Class names use `PascalCase`.
- Type hints are used throughout, as required by FastAPI for request and response validation.
- Each service module is kept in its own file under the `services/` directory, with a single responsibility.

TypeScript (Frontend):

- Code follows the **Airbnb TypeScript style guide** conventions.
- All React components are written as functional components with typed props.

- Component names use `PascalCase`. Variables and functions use `camelCase`.
- Each component is kept in its own file under the `components/` directory.
- API calls are made through Next.js API route proxies rather than directly from the browser, to avoid exposing backend URLs.

General:

- All sensitive configuration values such as database passwords and API keys are stored in environment variables and never hardcoded in source files.
- Git was used for version control throughout the project.

4.3 Laboratory Setup

DNS Guard was developed and tested on the following hardware and software environment:

Table 4.3: Laboratory Setup

Component	Details
Operating System	macOS
Processor	Apple Silicon (ARM64)
RAM	18 GB
Storage	1 TB SSD
Docker Version	Docker Desktop 4.x
Python Version	3.11
Node.js Version	20.x (LTS)
Browser for Testing	Google Chrome (latest)
Code Editor	Visual Studio Code
Version Control	Git with GitHub
LLM (Optional)	Ollama with Mistral 7B

All six services (frontend, backend, AI service, PostgreSQL, Redis, Qdrant) were run locally using Docker Compose during development. The system was tested by analysing a range of real and synthetic domain names, including known malicious domains from public threat feeds and legitimate domains from the Alexa top 1000 list.

4.4 Tools and Technology Used

The tools and technologies used in the implementation of DNS Guard are described in detail in Section 2.4. A brief summary is provided here for reference:

Table 4.4: Tools and Technologies Used

Tool / Technology	Version	Role in Project
Next.js	14	Frontend web application
Tailwind CSS	3.x	Frontend styling
FastAPI	0.110+	Backend API and AI service
Celery	5.x	Asynchronous task queue
PostgreSQL	15	Persistent data storage
Redis	7.x	Message broker for Celery
Qdrant	1.x	Vector database for RAG
scikit-learn	1.4+	ML model training and inference
dnspython	2.x	DNS record resolution
ReportLab	4.x	PDF report generation
Ollama	Latest	Local LLM runtime
Docker Compose	2.x	Service orchestration

4.5 Screenshots / Snapshots

This section presents screenshots of the DNS Guard web application showing the key features of the system.

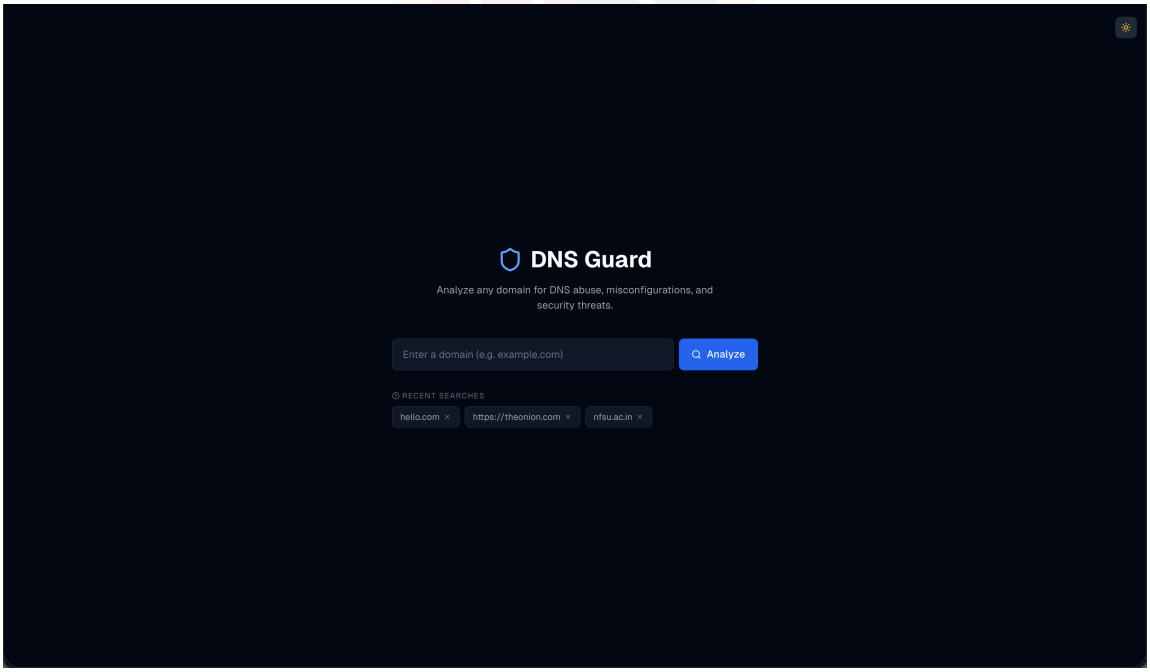

Figure 4.8: DNS Guard – Home Page with Search Interface

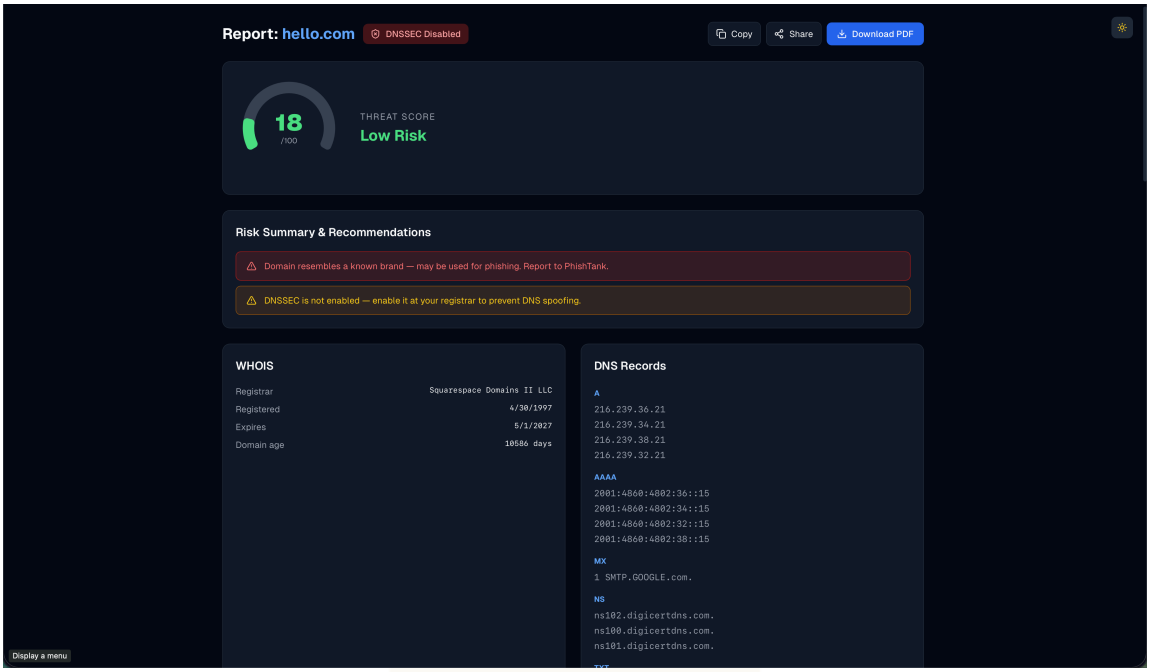

Figure 4.9: DNS Guard – Analysis Dashboard (Part 1: Threat Score and DNS Records)

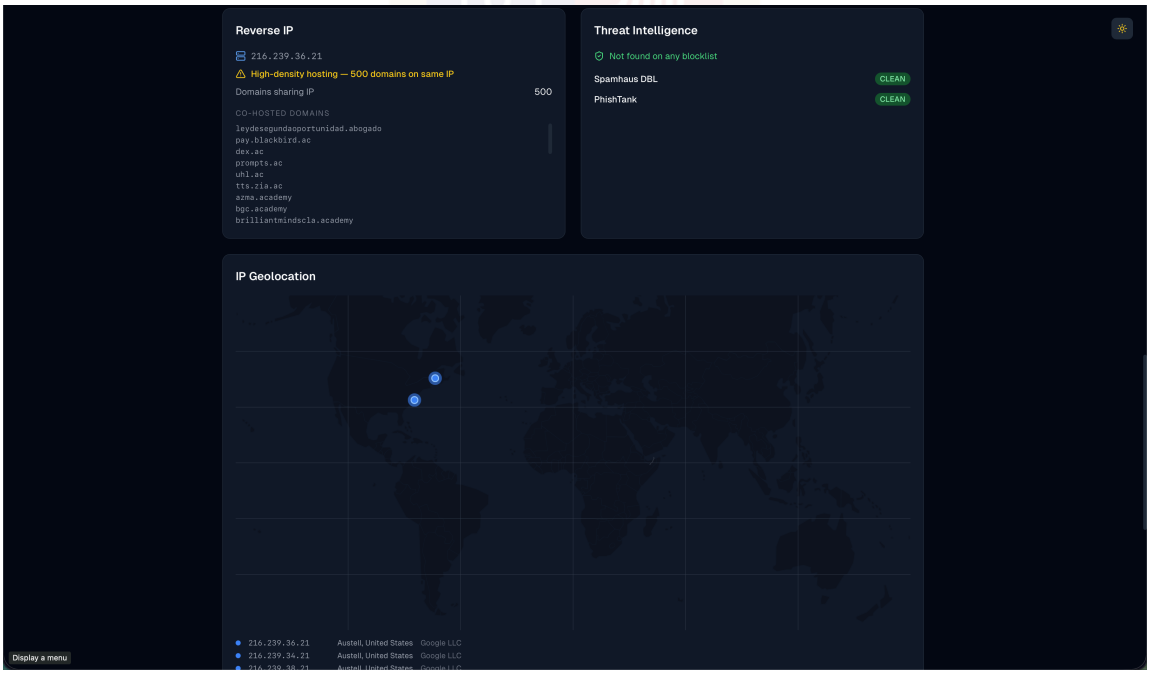

Figure 4.10: DNS Guard – Analysis Dashboard (Part 2: WHOIS and Certificate Transparency)

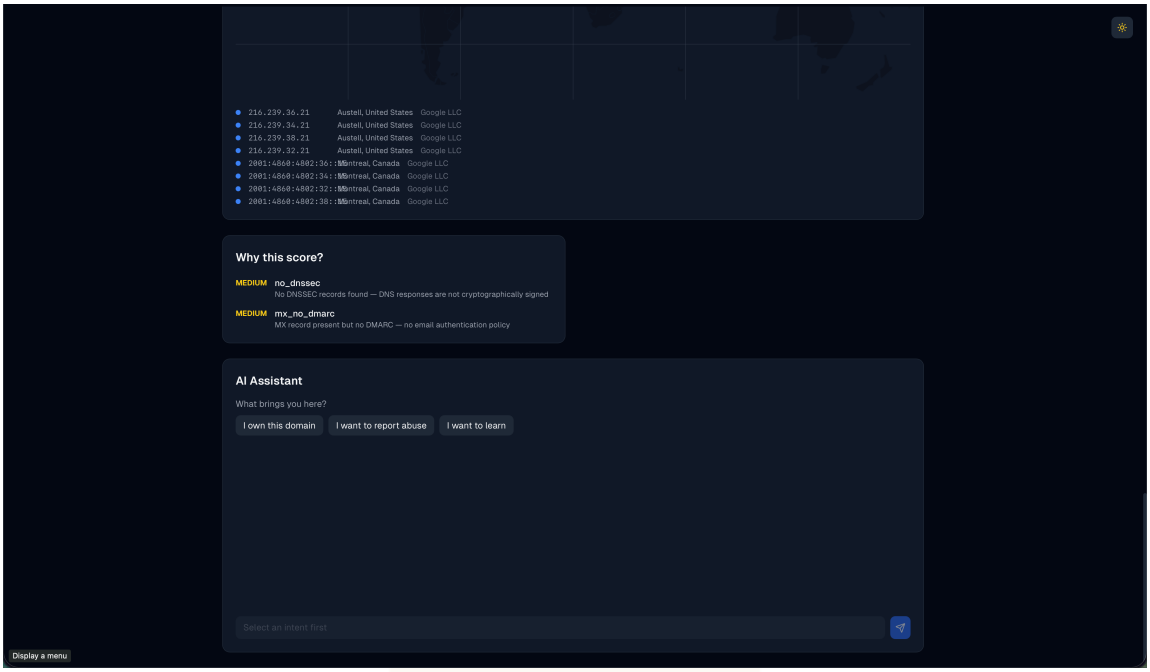

Figure 4.11: DNS Guard – Analysis Dashboard (Part 3: Threat Intelligence and Geolocation)

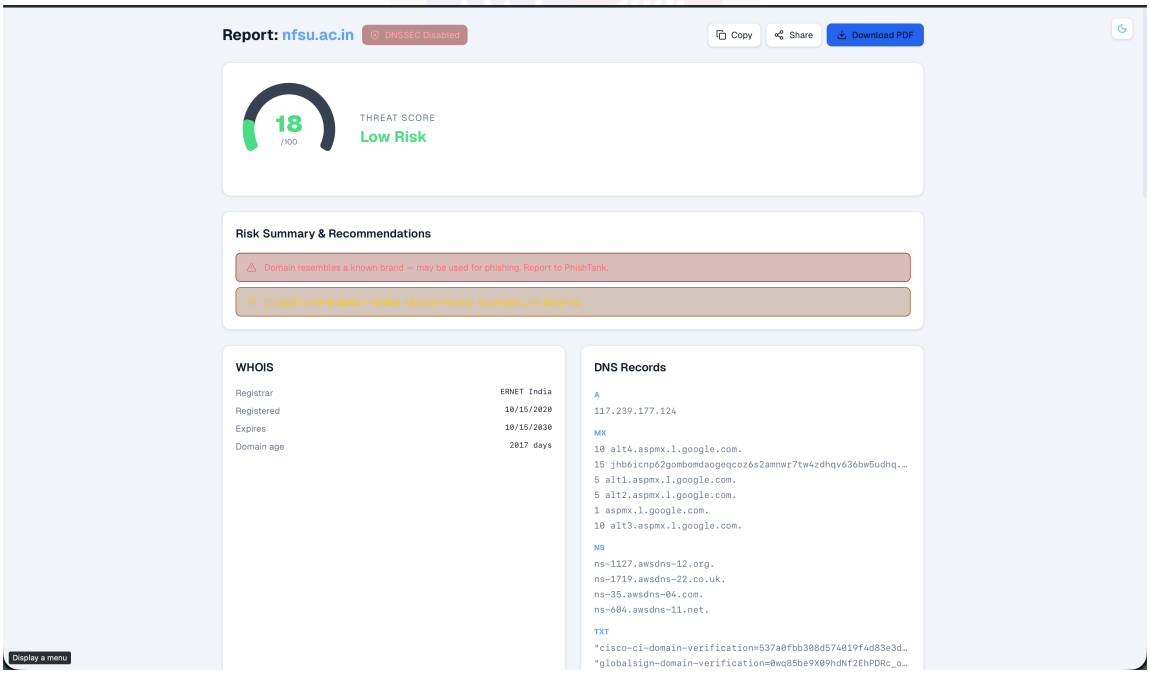

Figure 4.12: DNS Guard – Light Mode Interface

Summary of Results and Future Scope

5.1 Advantages and Unique Features

DNS Guard offers several advantages over existing tools and brings a number of unique features that are not commonly found together in a single free platform.

All-in-one analysis: Most existing tools focus on one area of DNS security. DNS Guard combines DNS record analysis, DNSSEC validation, WHOIS lookup with change detection, certificate transparency monitoring, passive DNS history, typosquatting detection, subdomain enumeration, reverse IP lookup, and threat intelligence checks into a single scan. The user does not need to visit multiple tools or manually combine results.

Machine learning threat score with explainability: DNS Guard produces a 0 to 100 threat score using an ensemble of a Random Forest classifier and an MLP neural network trained on 40 features. Alongside the score, the system shows a feature importance chart that explains which features contributed most to the score. This makes the result transparent and actionable, rather than just a black-box number.

WHOIS change detection: The system stores WHOIS data from every scan and compares it against the previous scan for the same domain. If any registration details have changed, such as the registrar, name servers, or contact information, the system highlights the differences. This is useful for detecting domain hijacking or unauthorised transfers.

AI chatbot with RAG: The built-in chatbot uses a local LLM (Mistral via Ollama) combined with a Retrieval-Augmented Generation pipeline backed by Qdrant. This means the chatbot can answer questions about DNS security using a curated knowledge base, rather than relying purely on the model's training data. A rule-based fallback engine ensures the chatbot remains useful even when Ollama is not running.

Privacy-first design: Because the LLM runs locally using Ollama, no user data is sent to any external AI service. This is an important advantage for users who are analysing sensitive or internal domains.

Free and open deployment: The entire system runs on Docker Compose and can be started with a single command. All core functionality works without any paid API keys, making it accessible to students, researchers, and small organisations.

PDF report generation: Each scan can be exported as a comprehensive PDF report that includes all findings, the threat score, feature importance, and remediation recommendations.

This is useful for sharing results with colleagues or including in security audits.

Dark and light mode: The interface supports both dark and light themes with a persistent toggle, making it comfortable to use in different environments.

5.2 Results and Discussions

The system was tested by analysing a range of domains across different risk categories. The test set included known malicious domains from public threat feeds, newly registered domains, domains with missing email security records, legitimate high-traffic domains, and domains that are typosquats of well-known brands.

Threat scoring accuracy: The ensemble model correctly assigned high threat scores (above 70) to domains that were present on blocklists or exhibited multiple risk indicators such as high lexical entropy, missing DNSSEC, and no SPF or DMARC records. Legitimate domains from the Alexa top 1000 consistently received low scores (below 30). The model performed well on domains with clear risk signals but was less confident on borderline cases, which is expected given the complexity of the problem.

DNSSEC validation: The system correctly identified domains with valid DNSSEC configurations and flagged those with missing or incomplete DNSKEY, RRSIG, or DS records. A large proportion of tested domains did not have DNSSEC enabled, which is consistent with published statistics showing that DNSSEC adoption remains low globally [32].

Typosquatting detection: The edit-distance based typosquatting detector successfully identified domains that were one or two character substitutions away from well-known brands. For example, domains such as `g00gle.com` and `paypal.com` were correctly flagged as potential typosquats.

Certificate transparency: The system successfully retrieved certificate data from crt.sh for all tested domains. Domains with a large number of recently issued certificates or wildcard certificates were flagged for further review.

Passive DNS and fast-flux detection: For domains with available passive DNS data, the system correctly identified fast-flux behaviour in domains that had resolved to a large number of different IP addresses over a short period.

Chatbot performance: When Ollama was running, the chatbot provided accurate and contextually relevant answers to questions about DNS security topics. The RAG pipeline ensured that answers were grounded in the knowledge base rather than being generated purely from the model's memory. The rule-based fallback also handled common questions correctly when Ollama was not available.

Performance: A typical full domain analysis completed in approximately 8 to 15 seconds, depending on the responsiveness of external services such as WHOIS servers and crt.sh. The asynchronous Celery architecture ensured that the API remained responsive during analysis and that multiple domains could be analysed concurrently.

5.3 Future Scope of Work

While DNS Guard is a functional and comprehensive platform, there are several areas where it could be improved or extended in future work.

Real-time monitoring and alerting: Currently, DNS Guard performs analysis on demand when a user submits a domain. A future version could include a monitoring mode where the system periodically re-scans a list of watched domains and sends an alert if the threat score changes significantly or if WHOIS data changes. This would be useful for organisations that need to monitor their own domains or track competitor domains.

Expanded threat intelligence feeds: The current implementation uses Spamhaus DBL and PhishTank. Future versions could integrate additional feeds such as the SURBL multi-blocklist [33], OpenPhish [34], and abuse.ch URLhaus [35] to improve detection coverage.

VirusTotal and Shodan integration: The system already has optional support for VirusTotal and Shodan API keys. Future work could make fuller use of these APIs, for example by retrieving VirusTotal scan history for a domain and Shodan data about the services running on its IP addresses.

Improved machine learning model: The current model uses a 40-feature vector with a Random Forest and MLP ensemble. Future work could explore deep learning approaches such as character-level convolutional neural networks or transformer-based models for domain name classification, which have shown strong results in recent research [16]. The training dataset could also be expanded and kept up to date with fresh malicious domain samples.

User accounts and scan history: The current system stores scan results in the database but does not have user accounts. Adding authentication would allow users to log in, view their full scan history, set up domain watchlists, and manage alerts.

API access: Exposing a public REST API with rate limiting and API key authentication would allow other tools and scripts to query DNS Guard programmatically, making it useful as a backend service for other security platforms.

Automated remediation guidance: The current system provides a risk summary with recommendations. Future work could make these recommendations more specific and actionable, for example by generating ready-to-use DNS record configurations that the user can copy and paste into their DNS provider's control panel.

Support for internationalised domain names (IDNs): Internationalised domain names use Unicode characters and are increasingly used in homograph attacks, where characters from different scripts are used to create domains that look identical to legitimate ones. Future versions of DNS Guard could add specific detection for IDN-based attacks [3].

Mobile-friendly interface: While the current interface is responsive, it was primarily designed for desktop use. A dedicated mobile-optimised layout would improve usability on smartphones and tablets.

Conclusion

This project set out to build a comprehensive, accessible, and free platform for analysing the security of domain names. The result is DNS Guard, a web-based domain threat intelligence platform that brings together multiple areas of DNS security analysis into a single tool.

The system successfully meets all the objectives defined at the start of the project. It can analyse any domain name and check it across ten different dimensions: DNS record analysis, DNSSEC validation, WHOIS lookup with change detection, certificate transparency monitoring, passive DNS history with fast-flux detection, typosquatting detection, subdomain enumeration, reverse IP lookup, and threat intelligence blocklist checks. All of these results are presented in a clean and easy-to-understand dashboard that works for both technical and non-technical users.

The machine learning component of the project produced a working threat scoring model that uses a 40-feature vector and an ensemble of a Random Forest classifier and an MLP neural network. The model assigns a risk score between 0 and 100 to any domain and explains which features contributed most to the score using a SHAP-style feature importance chart. This level of explainability is important because it allows users to understand why a domain was flagged and what they can do about it.

The AI chatbot, powered by a local Ollama LLM with a RAG pipeline backed by Qdrant, provides a conversational interface for asking questions about DNS security and about the results of a scan. Running the LLM locally means that no user data is sent to any external service, which is an important consideration for privacy.

The system is built as a set of containerised microservices using Docker Compose, which makes it easy to deploy and run on any machine with a single command. The use of FastAPI, Celery, PostgreSQL, and Redis provides a solid and scalable foundation that could support a much larger number of users with relatively minor changes.

From a broader perspective, this project demonstrates that it is possible to build a capable and practical DNS security tool using entirely open-source technologies at no cost. The fragmentation of existing tools, where a user must visit five or six different websites to get a complete picture of a domain's security, is a real problem that DNS Guard addresses directly.

The project also highlights how important DNS security is and how many domains on the internet still have basic misconfigurations such as missing SPF, DKIM, and DMARC records, no DNSSEC, and recently registered names that closely resemble well-known brands. These

are not difficult problems to fix, but they require awareness. DNS Guard is designed to provide that awareness in a clear and actionable way.

In conclusion, DNS Guard is a functional, well-structured, and practically useful final year project that covers a wide range of topics including networking, cybersecurity, machine learning, natural language processing, and full-stack web development. The work done here provides a strong foundation for future development and could, with further hardening and testing, be deployed as a real-world security tool.

References

[1] P. Mockapetris, "Domain names – concepts and facilities," Internet Engineering Task Force, RFC 1034, 1987. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1034

[2] S. Banerjee, S. Bhatt, B. Krishnamurthy, and M. Mah, "Detecting typosquatting domains," in *Proceedings of the 2008 ACM Workshop on Recurring Malcode*, ACM, 2008, pp. 1–8.

[3] T. Holz, C. Gorecki, K. Rieck, and F. C. Freiling, "Measuring and detecting fast-flux service networks," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, Internet Society, 2008.

[4] S. Kitterman, "Sender policy framework (spf) for authorizing use of domains in email," Internet Engineering Task Force, RFC 7208, 2014. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7208

[5] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, "Dns security introduction and requirements," Internet Engineering Task Force, RFC 4033, 2005. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4033

[6] Anti-Phishing Working Group, "Phishing activity trends report, q4 2023," APWG, Tech. Rep., 2023. [Online]. Available: https://apwg.org/trendsreports/

[7] VirusTotal, *Virustotal – free online virus, malware and url scanner*, Accessed: April 2026, 2024. [Online]. Available: https://www.virustotal.com

[8] MXToolbox, *Mxtoolbox – email header analyzer, mx record lookup*, Accessed: April 2026, 2024. [Online]. Available: https://mxtoolbox.com

[9] Shodan, *Shodan – the search engine for the internet of everything*, Accessed: April 2026, 2024. [Online]. Available: https://www.shodan.io

[10] SecurityTrails, *Securitytrails – dns intelligence and historical data*, Accessed: April 2026, 2024. [Online]. Available: https://securitytrails.com

[11] HackerTarget, *Dnsdumpster – dns recon and research*, Accessed: April 2026, 2024. [Online]. Available: https://dnsdumpster.com

[12] PhishTank, *Phishtank – free community site for phishing data*, Accessed: April 2026, 2024. [Online]. Available: https://www.phishtank.com

[13] Spamhaus, *The spamhaus project – domain block list*, Accessed: April 2026, 2024. [Online]. Available: https://www.spamhaus.org

[14] M. Antonakakis, R. Perdisci, W. Lee, N. Vasiloglou, and D. Dagon, "Detecting malware domains at the upper dns hierarchy," in *Proceedings of the 20th USENIX Security Symposium*, USENIX Association, 2011.

[15] L. Bilge, E. Kirda, C. Kruegel, and M. Balduzzi, "Exposure: Finding malicious domains using passive dns analysis," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, Internet Society, 2011.

[16] J. Woodbridge, H. S. Anderson, A. Ahuja, and D. Grant, "Predicting domain generation algorithms with long short-term memory networks," in *arXiv preprint arXiv:1611.00791*, 2016.

[17] Z. C. Lipton, "The mythos of model interpretability," *Queue*, vol. 16, no. 3, pp. 31–57, 2018. DOI: 10.1145/3236386.3241340

[18] B. Halley and B. Westenberg, *Dnspython – a dns toolkit for python*, Accessed: April 2026, 2024. [Online]. Available: https://www.dnspython.org

[19] F. Pedregosa et al., "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[20] S. Ramirez, *Fastapi – modern, fast web framework for building apis with python*, Accessed: April 2026, 2024. [Online]. Available: https://fastapi.tiangolo.com

[21] Vercel, *Next.js – the react framework for the web*, Accessed: April 2026, 2024. [Online]. Available: https://nextjs.org

[22] Docker Inc., *Docker – accelerated container application development*, Accessed: April 2026, 2024. [Online]. Available: https://www.docker.com

[23] The PostgreSQL Global Development Group, *Postgresql – the world's most advanced open source relational database*, Accessed: April 2026, 2024. [Online]. Available: https://www.postgresql.org

[24] Redis Ltd., *Redis – the real-time data platform*, Accessed: April 2026, 2024. [Online]. Available: https://redis.io

[25] Qdrant, *Qdrant – vector database for the next generation of ai applications*, Accessed: April 2026, 2024. [Online]. Available: https://qdrant.tech

[26] Ollama, *Ollama – get up and running with large language models locally*, Accessed: April 2026, 2024. [Online]. Available: https://ollama.com

[27] A. Q. Jiang et al., "Mistral 7b," *arXiv preprint arXiv:2310.06825*, 2023.

[28] Tailwind Labs, *Tailwind css – a utility-first css framework*, Accessed: April 2026, 2024. [Online]. Available: `https://tailwindcss.com`

[29] A. Solem and contributors, *Celery – distributed task queue*, Accessed: April 2026, 2024. [Online]. Available: `https://docs.celeryq.dev`

[30] ReportLab Inc., *Reportlab – pdf generation for python*, Accessed: April 2026, 2024. [Online]. Available: `https://www.reportlab.com`

[31] G. van Rossum, B. Warsaw, and N. Coghlan, *Pep 8 – style guide for python code*, Accessed: April 2026, 2001. [Online]. Available: `https://peps.python.org/pep-0008/`

[32] APNIC, *Dnssec validation statistics*, Accessed: April 2026, 2023. [Online]. Available: `https://stats.labs.apnic.net/dnssec`

[33] SURBL, *Surbl – multi-blocklist for uri reputation*, Accessed: April 2026, 2024. [Online]. Available: `https://www.surbl.org`

[34] OpenPhish, *Openphish – phishing intelligence*, Accessed: April 2026, 2024. [Online]. Available: `https://openphish.com`

[35] abuse.ch, *Urlhaus – malware url exchange*, Accessed: April 2026, 2024. [Online]. Available: `https://urlhaus.abuse.ch`

Chapter A

API Reference

This appendix lists all REST API endpoints exposed by the DNS Guard backend. All endpoints are served from the base URL `http://localhost:8000`. Requests and responses use JSON unless otherwise stated.

Table A.1: API Endpoint Reference

Method	Endpoint	Description
POST	`/analyze`	Full domain analysis. Accepts `{"domain": "example.com"}`. Returns DNS records, WHOIS, certs, passive DNS, typosquat matches, subdomains, reverse IP, threat intel, geolocation, threat score, verdict, and feature explanations.
POST	`/chat`	AI chatbot message. Accepts `{"domain": "...", "intent": "...", "message": "..."}`. Returns a text response from the LLM or rule-based fallback.
GET	`/report/{domain}/pdf`	Downloads a PDF report for the given domain. Triggers a fresh analysis and returns the result as an `application/pdf` file attachment.
GET	`/whois/{domain}`	Returns WHOIS registration data including registrar, creation date, expiry date, and name servers.

Method	Endpoint	Description
GET	`/certs/{domain}`	Returns certificate transparency data from crt.sh including certificate count, issuer list, and wildcard certificate flag.
GET	`/passive-dns/{domain}`	Returns historical IP resolution data and a fast-flux detection flag.
GET	`/typosquat/{domain}`	Returns a list of brand domains similar to the submitted domain with edit-distance similarity scores.
GET	`/subdomains/{domain}`	Returns a list of discovered subdomains from brute-force enumeration and certificate transparency logs.
GET	`/reverse-ip/{domain}`	Returns the resolved IP address and a list of other domains co-hosted on the same IP.
GET	`/threat-intel/{domain}`	Returns blocklist check results from Spamhaus DBL and PhishTank.

Chapter B

Machine Learning Model Details

B.1 Feature List

The threat scoring model uses a 40-dimensional feature vector extracted from the domain name and its DNS records.

Table B.1: ML Model Feature Vector (40 Features)

No.	Feature Name	Type	Description
0	`domain_length`	Integer	Total length of the full domain name string.
1	`sld_length`	Integer	Length of the second-level domain.
2	`subdomain_depth`	Integer	Number of dots in the domain.
3	`hyphen_count`	Integer	Number of hyphens in the domain name.
4	`sld_entropy`	Float	Shannon entropy of the SLD characters. High entropy suggests DGA-generated names.
5	`digit_ratio`	Float	Ratio of digit characters to total characters in the SLD.
6	`consonant_ratio`	Float	Ratio of consonant characters in the SLD.
7	`long_digit_seq`	Binary	1 if the SLD contains 4 or more consecutive digits.
8	`high_risk_tld`	Binary	1 if the TLD is in the high-risk list (e.g. `.tk`, `.xyz`, `.top`).
9	`brand_impersonation`	Binary	1 if the SLD contains the name of a well-known brand.
10	`phish_keywords`	Binary	1 if the SLD contains keywords such as `login`, `secure`, `verify`.

No.	Feature Name	Type	Description
11	`spam_keywords`	Binary	1 if the SLD contains keywords such as `free`, `win`, `prize`.
12	`has_a_record`	Binary	1 if the domain has at least one A record.
13	`has_aaaa_record`	Binary	1 if the domain has at least one AAAA record.
14	`has_mx_record`	Binary	1 if the domain has at least one MX record.
15	`ns_count`	Integer	Number of NS records for the domain.
16	`a_record_count`	Integer	Number of A records. A high count is a fast-flux indicator.
17	`has_soa`	Binary	1 if the domain has a SOA record.
18	`has_cname`	Binary	1 if the domain has a CNAME record.
19	`spf_present`	Binary	1 if a valid SPF record is found in TXT records.
20	`dmarc_present`	Binary	1 if a valid DMARC record is found in TXT records.
21	`dkim_present`	Binary	1 if a DKIM record is found in TXT records.
22	`spf_includes`	Binary	1 if the SPF record uses `include:` directives.
23	`fast_flux`	Binary	1 if the domain has more than 5 A records.
24	`many_ns`	Binary	1 if the domain has more than 6 NS records.
25	`no_mx_has_a`	Binary	1 if the domain has an A record but no MX record.
26	`dnskey_present`	Binary	1 if a DNSKEY record is present.
27	`rrsig_present`	Binary	1 if an RRSIG record is present.
28	`ds_present`	Binary	1 if a DS record is present.
29	`no_dnssec`	Binary	1 if neither DNSKEY nor RRSIG is present.
30	`mx_no_spf`	Binary	1 if the domain has MX records but no SPF record.
31	`mx_no_dmarc`	Binary	1 if the domain has MX records but no DMARC record.

No.	Feature Name	Type	Description
32	`very_long_sld`	Binary	1 if the SLD is longer than 20 characters.
33	`high_entropy_sld`	Binary	1 if the SLD entropy is greater than 3.5.
34	`deep_subdomain`	Binary	1 if the domain has more than 3 dots.
35	`brand_in_subdomain`	Binary	1 if a brand name appears in a subdomain but not in the SLD.
36–39	`reserved_36--39`	Float	Reserved for future features. Currently 0.0.

B.2 Training Data Summary

The model was trained on a labelled dataset compiled from the following sources:

- **Malicious domains:** Spamhaus DBL, PhishTank, and abuse.ch URLhaus feeds.
- **Legitimate domains:** Alexa top 1 million domain list, filtered to remove any domains present in threat feeds.

The dataset was balanced to contain an approximately equal number of malicious and legitimate samples.

B.3 Model Evaluation

Table B.2: ML Model Evaluation Metrics

Metric	Value
Accuracy	To be updated after final training
Precision	To be updated after final training
Recall	To be updated after final training
F1 Score	To be updated after final training
AUC-ROC	To be updated after final training

Chapter C

Sample Scan Output

The following is a representative sample of the JSON response returned by the `POST /analyze` endpoint for the domain `example.com`.


```
{
  "domain": "example.com",
  "threat_score": 12.5,
  "verdict": "Low Risk",
  "explanations": [
    { "feature": "no_dnssec", "value": 1.0, "contribution": 8.2 },
    { "feature": "mx_no_dmarc", "value": 1.0, "contribution": 4.3 }
  ],
  "dns_records": {
    "A": ["93.184.216.34"],
    "NS": ["a.iana-servers.net", "b.iana-servers.net"],
    "MX": [],
    "TXT": ["v=spf1 -all"],
    "SOA": ["ns.icann.org. noc.dns.icann.org. ..."]
  },
  "dnssec": { "dnskey": true, "rrsig": true, "ds": true },
  "whois": {
    "registrar": "ICANN",
    "creation_date": "1995-08-14",
    "expiry_date": "2025-08-13",
    "name_servers": ["A.IANA-SERVERS.NET", "B.IANA-SERVERS.NET"]
  },
  "certs": { "count": 4, "has_wildcard": false,
             "issuers": ["DigiCert Inc", "Let's Encrypt"] },
  "passive_dns": { "ips": ["93.184.216.34"], "fast_flux": false },
  "typosquat": { "matches": [] },
```

```
  "subdomains": { "found": ["www.example.com", "mail.example.com"] },
  "reverse_ip": {
    "ip": "93.184.216.34",
    "domains": ["example.com", "example.net", "example.org"]
  },
  "threat_intel": { "spamhaus_dbl": false, "phishtank": false },
  "geo": [{
    "ip": "93.184.216.34", "country": "United States",
    "city": "Norwell", "lat": 42.1596, "lon": -70.8217
  }]
}
```

Chapter D

Environment Variables and Configuration

All environment variables are defined in the `.env` file at the root of the project. A template is provided in `.env.example`.

Table D.1: Environment Variables and Configuration

Variable	Default Value	Description
POSTGRES_USER	dnsabuse	PostgreSQL database username.
POSTGRES_PASSWORD	changeme	PostgreSQL password. Change in production.
POSTGRES_DB	dnsabuse_db	Name of the PostgreSQL database.
DATABASE_URL	postgresql://...	Full SQLAlchemy connection string.
REDIS_URL	redis://redis:6379/0	Redis connection URL for Celery.
QDRANT_URL	http://qdrant:6333	URL of the Qdrant vector database.
AI_SERVICE_URL	http://ai:8001	Internal URL of the AI/ML service.
OLLAMA_BASE_URL	http://host.docker.internal:11434	URL of the Ollama LLM on the host machine.
OLLAMA_MODEL	mistral	Ollama model name for the chatbot.
VIRUSTOTAL_API_KEY	(empty)	Optional VirusTotal API key.
SHODAN_API_KEY	(empty)	Optional Shodan API key.
SECRET_KEY	changeme	App secret key. Must be changed in production.

Variable	Default Value	Description
`ENVIRONMENT`	`development`	Set to `production` to disable debug features.

Chapter E

Installation and Deployment Guide

E.1 Prerequisites

- **Docker Desktop** (v4.0+) – `https://www.docker.com/products/docker-desktop`
- **Git** – `https://git-scm.com`
- (Optional) **Ollama** – `https://ollama.com`

E.2 Installation Steps

Step 1: Clone the repository

```
git clone <repository-url>
cd FYP
```

Step 2: Configure environment variables

```
cp .env.example .env
```

Open `.env` and change `POSTGRES_PASSWORD` and `SECRET_KEY` for any non-development use.

Step 3: (Optional) Set up Ollama

```
ollama pull mistral
```

Ollama must be running on the host machine before starting Docker.

Step 4: Build and start all services

```
docker compose up --build -d
```

This starts six containers: frontend, backend, AI service, PostgreSQL, Redis, and Qdrant. The first build may take several minutes.

Step 5: Open the application

```
http://localhost:3000
```

E.3 Stopping the Application

```
docker compose down
```

To also remove all stored data:

```
docker compose down -v
```

E.4 Updating Threat Feeds

```
python scripts/update_feeds.py
```

Chapter F

Sample Generated PDF Report

The following pages contain a sample PDF report generated by DNS Guard for the domain `nfsu.ac.in`. This report is produced by the `GET /report/{domain}/pdf` endpoint and is generated using the ReportLab library.

DNS Guard

Domain Threat Intelligence Report

18 /100

Low Risk

Domain: nfsu.ac.in

Generated: 2026-04-24 16:33 UTC

Risk Summary & Key Findings

MEDIUM	**No Dnssec** — No DNSSEC records found — DNS responses are not cryptographically signed
MEDIUM	**Mx No Dmarc** — MX record present but no DMARC — no email authentication policy

WHOIS Information

Registrar	ERNET India
Created	2020-10-15
Expires	2030-10-15
Domain age	2017 days
Days to expiry	1634 days
New domain	✗ No
Expiring soon	✗ No

DNS Records

A	117.239.177.124
MX	10 alt4.aspmx.l.google.com. 15 jhb6icnp62gombomdaogeqcoz6s2amnwr7tw4zdhqv636bw5udhq.mx-verification.google.com. 5 alt1.aspmx.l.google.com. 5 alt2.aspmx.l.google.com. 1 aspmx.l.google.com. 10 alt3.aspmx.l.google.com.
NS	ns-1127.awsdns-12.org. ns-1719.awsdns-22.co.uk. ns-35.awsdns-04.com. ns-604.awsdns-11.net.

TXT	"cisco-ci-domain-verification=537a0fbb308d574019f4d83e3d097d428744ef328c41c9f037e09325d5c135dc" "globalsign-domain-verification=0wq85be9X09hdNf2EhPDRc_oXvg7c31uXQX_L4pm6L" "google-site-verification=zsUfE6K5vqjrC_2KKD5PGjB46N9AlscmBmcv0Ze2vBM" "v=spf1 include:_spf.leewaysofttech.com include:_spf.google.com include:spf.mandrillapp.com include:spf.smtpconnect.com ~all" "Sendinblue-code:a8581e01263ce2669e381130c08a8625"
SOA	ns-604.awsdns-11.net. awsdns-hostmaster.amazon.com. 1 7200 900 1209600 86400

DNSSEC Status

✗ **DNSSEC Not Configured**

DNSKEY	✗ No
RRSIG	✗ No
DS	✗ No

Certificate Transparency

Total certificates	—
Wildcard certificates	—
Issued last 30 days	—
Certificate spike	✗ No
Has wildcards	✗ No

Passive DNS

Total records	5
Unique IPs seen	5
Fast-flux suspected	✗ No

IP Address	Seen
117.239.183.26	1
117.239.177.124	1
103.118.16.128	1
210.212.128.87	1
13.126.225.76	1

Typosquat Detection

■ **Possible typosquat detected**

Target Domain	Edit Distance
abc.es	1
1c.ru	1
as.com	1
ad.nl	1
lc.chat	1

Subdomain Enumeration

Total found	41
Suspicious	1

Suspicious subdomains:

- `login.nfsu.ac.in`

Reverse IP Lookup

Resolved IP	117.239.177.124
Domains on same IP	18
High-density host	✗ No

Co-hosted domains:

- `beta.nfsu.ac.in`
- `bhopal.nfsu.ac.in`
- `bhubaneswar.nfsu.ac.in`
- `chennai.nfsu.ac.in`
- `delhi.nfsu.ac.in`
- `dfeg.nfsu.ac.in`
- `dharwad.nfsu.ac.in`
- `goa.nfsu.ac.in`
- `gujarat.nfsu.ac.in`
- `guwahati.nfsu.ac.in`

Threat Intelligence

✓ **Not found on any blocklist**

Spamhaus DBL	Clean
PhishTank	Clean

Generated by DNS Guard · 2026-04-24 16:33 UTC · For security research purposes only.

List of Publications / Online References

Online References

1. P. Mockapetris, "Domain Names – Concepts and Facilities," RFC 1034, Internet Engineering Task Force, 1987. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1034

2. R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, "DNS Security Introduction and Requirements," RFC 4033, IETF, 2005. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4033

3. S. Kitterman, "Sender Policy Framework (SPF)," RFC 7208, IETF, 2014. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7208

4. Anti-Phishing Working Group, "Phishing Activity Trends Report, Q4 2023," APWG, 2023. [Online]. Available: https://apwg.org/trendsreports/

5. VirusTotal, "Free Online Virus, Malware and URL Scanner." [Online]. Available: https://www.virustotal.com

6. MXToolbox, "Email Header Analyzer, MX Record Lookup." [Online]. Available: https://mxtoolbox.com

7. Shodan, "The Search Engine for the Internet of Everything." [Online]. Available: https://www.shodan.io

8. SecurityTrails, "DNS Intelligence and Historical Data." [Online]. Available: https://securitytrails.com

9. HackerTarget, "DNSdumpster – DNS Recon and Research." [Online]. Available: https://dnsdumpster.com

10. PhishTank, "Free Community Site for Phishing Data." [Online]. Available: https://www.phishtank.com

11. The Spamhaus Project, "Domain Block List (DBL)." [Online]. Available: https://www.spamhaus.org

12. SURBL, "Multi-Blocklist for URI Reputation." [Online]. Available: https://www.surbl.org

13. OpenPhish, "Phishing Intelligence." [Online]. Available: https://openphish.com

14. abuse.ch, "URLhaus – Malware URL Exchange." [Online]. Available: https://urlhaus.abuse.ch

15. APNIC, "DNSSEC Validation Statistics," 2023. [Online]. Available: https://stats.labs.apnic.net/dnssec

16. Vercel, "Next.js – The React Framework for the Web." [Online]. Available: https://nextjs.org

17. Tailwind Labs, "Tailwind CSS – A Utility-First CSS Framework." [Online]. Available: https://tailwindcss.com

18. S. Ramirez, "FastAPI – Modern, Fast Web Framework for Building APIs with Python." [Online]. Available: https://fastapi.tiangolo.com

19. Ask Solem et al., "Celery – Distributed Task Queue." [Online]. Available: https://docs.celeryq.dev

20. The PostgreSQL Global Development Group, "PostgreSQL – The World's Most Advanced Open Source Relational Database." [Online]. Available: https://www.postgresql.org

21. Redis Ltd., "Redis – The Real-Time Data Platform." [Online]. Available: https://redis.io

22. Qdrant, "Vector Database for the Next Generation of AI Applications." [Online]. Available: https://qdrant.tech

23. Ollama, "Get Up and Running with Large Language Models Locally." [Online]. Available: https://ollama.com

24. B. Halley and B. Westenberg, "dnspython – A DNS Toolkit for Python." [Online]. Available: https://www.dnspython.org

25. ReportLab Inc., "ReportLab – PDF Generation for Python." [Online]. Available: https://www.reportlab.com

26. Docker Inc., "Docker – Accelerated Container Application Development." [Online]. Available: https://www.docker.com

27. G. van Rossum, B. Warsaw, and N. Coghlan, "PEP 8 – Style Guide for Python Code," 2001. [Online]. Available: https://peps.python.org/pep-0008/

28. ICANN, "ICANN – Internet Corporation for Assigned Names and Numbers." [Online]. Available: https://www.icann.org

29. ICANN, "DNS Abuse – ICANN's Role and Efforts to Address DNS Abuse." [Online]. Available: https://www.icann.org/dns-abuse

30. ICANN, "Registration Data Access Protocol (RDAP)." [Online]. Available: https://www.icann.org/rdap

31. DNS Abuse Institute, "NetBeacon – Reporting DNS Abuse." [Online]. Available: https://dnsabuseinstitute.org

32. ICANN Security and Stability Advisory Committee (SSAC), "SAC115 – SSAC Advisory on DNS Abuse," ICANN, 2021. [Online]. Available: https://www.icann.org/en/system/files/files/sac-115-en.pdf

33. M3AAWG and ICANN, "DNS Abuse Mitigation – Best Practices for Registrars and Registries," 2022. [Online]. Available: https://www.icann.org/en/blogs/details/dns-abuse-mitigation

Publications

No publications have been produced from this work at the time of submission. The project is an academic final year dissertation submitted to the National Forensic Sciences University, Delhi Campus, in partial fulfilment of the B.Tech.-M.Tech. degree in Computer Science and Engineering (Cybersecurity).